

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

FINAL YEAR PROJECT REPORT
BACHELOR OF ENGINEERING

AFDM Receiver Design for Underwater Acoustic Communication: From Joint OMP-MP Baselines to Frozen Decision-Level Correction

Student:	Yikun GUO
GUID:	2840210g
UESTC Student No.:	2022190908014
1st Supervisor:	Yusha Liu
2nd Supervisor:	

2025–2026

Coursework Declaration and Feedback Form

This student should complete and sign this part.

Student Name: Yikun GUO	Student GUID: 2840210g
Course Code: UESTC4006P	Course Name: INDIVIDUAL PROJECT 4
UESTC No.: 2022190908014	Academic Year: 2025–2026
Name of 1st Supervisor: Yusha Liu	Name of 2nd Supervisor:
Title of Project: **AFDM Receiver Design for Underwater Acoustic Communication: From Joint OMP-MP Baselines to Frozen Decision-Level Correction**	
Declaration of Originality and Submission Information	
I affirm that this submission is all my own work in accordance with the University of Glasgow Regulations and the School of Engineering requirements. *Signed (Student):*	UESTC4006P TEC-IT.COM
Date of Submission:	
Feedback from Lecturer to Student - to be completed by Lecturer or Demonstrator	
Grade Awarded: Feedback (as appropriate to the coursework which was assessed):	
Lecturer / Demonstrator:	Date returned to the Teaching Office:

Abstract

This final report summarizes a two-stage AFDM receiver project for underwater acoustic communication. The midterm stage established a MATLAB end-to-end AFDM-UWA simulation chain and a joint OMP-MP receiver that used one-shot OMP support locking followed by iterative message passing and least-squares gain refinement. For consistency, the final report does not mix those MATLAB figures with newer Python experiments; instead, all numerical claims are unified to the latest Python BER protocol. Under that protocol, the strongest traditional receiver from the joint OMP family is the strict traditional scaffold TRADPARAM FIXED, which improves the historical JOINTOMPMPbaseline at 15, 20, and 25 dB. The later project stage then revisited learned detector-side improvement. After correcting the evaluation pipeline, the original mean-residual distillation story collapsed, showing that the real recoverable headroom lies in sparse decision-boundary corrections rather than smooth residual regression. Guided by this diagnosis, the final method is a lightweight frozen direct-bit corrector that uses observable receiver trajectory features only. Trained once and frozen, it improves the mean BER from 6.3553×10^{-2} to 6.2493×10^{-2} on untouched nominal caches, remains positive under higher Doppler severity, integer-Doppler structural shift, and lower pilot power, and yields relative goodput gains up to 11.33% for 256-bit packets. A severe path-count shift remains an explicit failure boundary. The report therefore presents a coherent project trajectory: a strong traditional midterm baseline, a corrected negative result on residual distillation, and a final scope-bounded post-hoc correction method with reproducible gains.

Acknowledgements

The author thanks Yusha Liu for supervision, feedback on the research direction, and guidance on how to convert the project from a promising midterm prototype into a final report with corrected and reproducible evidence.

Contents

Coursework Declaration and Feedback Form i

Abstract ii

Acknowledgements iii

1 Introduction and Project Scope 1
- 1.1 Motivation: the challenge of underwater acoustic communication . . . 1
- 1.2 Project scope and research questions . . . 1
- 1.3 Project contributions . . . 2
- 1.4 Results preview . . . 3
- 1.5 What this project is not . . . 3
- 1.6 Chapter roadmap . . . 3
- 1.7 Report structure . . . 3

2 Background and System Model 5
- 2.1 Doubly selective channels and the underwater acoustic setting . . . 5
- 2.2 AFDM waveform and receiver chain . . . 6
- 2.3 Sparse channel estimation: orthogonal matching pursuit . . . 6
- 2.4 Iterative detection: message passing and gain refinement . . . 7
- 2.5 Learned receiver extensions . . . 8
- 2.6 Notation summary . . . 9
- 2.7 Chapter summary . . . 9

3 Midterm Traditional AFDM Receiver 10
- 3.1 MATLAB simulation chain and midterm contribution . . . 10
- 3.2 One-shot OMP support locking . . . 11
- 3.3 Iterative MP detection and gain refinement . . . 11
- 3.4 Three traditional receiver variants . . . 12
- 3.5 Strengths and limitations of the traditional line . . . 13
- 3.6 Chapter summary . . . 14

4 Evaluation Protocol Correction and Diagnostic Pivot 15
- 4.1 Transition to the corrected Python evaluation . . . 15
- 4.2 Why the original mean-residual distillation story collapsed . . . 15
- 4.3 Decision-oracle analysis: the structure of recoverable headroom . . . 16
- 4.4 Linear gating: too weak for stable gains . . . 17
- 4.5 Design implication: toward decision-level correction . . . 18
- 4.6 Chapter summary . . . 18

5 Final Method: Frozen Decision-Level Corrector 20
- 5.1 High-level design . . . 20
- 5.2 Observable feature vector . . . 20
- 5.3 Model architecture . . . 22
- 5.4 Train-once / test-on-many protocol . . . 23
- 5.5 Complexity and overhead analysis . . . 23
- 5.6 Chapter summary . . . 24

6 Experimental Results **25**
6.1 Experimental setup . . . 25
6.2 Traditional baseline comparison . . . 25
6.3 Nominal train-once / test-on-many result . . . 26
6.4 Regime-shift transfer . . . 27
6.5 Explicit failure boundary at $P_{\text{paths}} = 15$ . . . 28
6.6 Feature ablation . . . 28
6.7 Correction-behaviour analysis . . . 29
6.8 Packet-level utility . . . 30
6.9 Chapter summary . . . 31

7 Discussion and Limitations **37**
7.1 Why the original residual-distillation story failed . . . 37
7.2 Why the frozen decision-level corrector works better . . . 37
7.3 Why the gains are modest but still meaningful . . . 38
7.4 Why $P_{\text{paths}} = 15$ is a true failure boundary . . . 38
7.5 Why a two-layer MLP rather than simpler classifiers . . . 39
7.6 Engineering cost and deployment realism . . . 40
7.7 Limitations . . . 41
7.8 Overall comparison of methods . . . 41
7.9 Chapter summary . . . 42

8 Conclusion and Future Work **43**
8.1 Project summary . . . 43
8.2 Limitations revisited . . . 43
8.3 Future work . . . 44
8.4 Closing remarks . . . 45

Appendices **47**

A Alignment of Midterm MATLAB Work and Final Python Reporting **47**
A.1 Midterm MATLAB artifacts . . . 47
A.2 Why MATLAB numbers are not reused in the final report . . . 47
A.3 Mapping midterm work to final results . . . 47

B Full Observable Feature Vector Definitions **47**

C Paired Bootstrap Confidence Interval Methodology **49**

D Additional BER Tables **49**

1 Introduction and Project Scope

1.1 Motivation: the challenge of underwater acoustic communication

Underwater acoustic (UWA) communication is one of the most demanding physical-layer environments known to communications engineers. The acoustic channel is simultaneously delay-dispersive and Doppler-dispersive. Delays arise from multipath propagation in the shallow-water waveguide, where reflections from the sea surface and seabed combine at the receiver with path lengths that differ by tens to hundreds of metres. Doppler effects arise from the motion of acoustic sources, receivers, and scatterers relative to the medium; even slow platform speeds produce significant Doppler shifts because the speed of sound in seawater (approximately 1500 m/s) is five orders of magnitude lower than the speed of light. For medium-range shallow-water links, typical channel parameters include delay spreads of 10–100 ms, maximum Doppler spreads of 1–10 Hz, and 3–15 significant propagation paths [1, 2].

These figures translate into a severe receiver design problem. A conventional OFDM system with a practical cyclic prefix length cannot absorb the full delay spread of a medium-range UWA channel without sacrificing spectral efficiency. Even if the prefix is long enough, the time-varying nature of the channel corrupts the orthogonality between subcarriers and creates inter-carrier interference that a standard single-tap frequency-domain equalizer cannot eliminate. High-mobility communication systems face a similar challenge: wideband terrestrial channels under vehicle speeds can produce Doppler-to-subcarrier-spacing ratios well above 0.01, at which point the standard OFDM pilot grid can no longer track the channel reliably.

Affine frequency division multiplexing (AFDM) was proposed in [3] as a waveform designed for doubly dispersive channels. AFDM encodes information on a set of chirp basis functions defined through the discrete affine Fourier transform (DAFT). By choosing the chirp parameters appropriately, AFDM can place all path components onto a structured discrete delay-Doppler-like grid, which supports sparse channel reasoning and allows model-driven receiver design. Unlike OTFS, which operates by explicitly transforming signals into the delay-Doppler domain, AFDM achieves a related effect through chirp-domain processing and tends to concentrate channel energy more favourably for channels with large Doppler-to-subcarrier-spacing ratios [4, 5].

Good waveform design is a necessary condition for reliable UWA communication, but it is not sufficient on its own. The receiver must still estimate the channel, detect the transmitted symbols, and manage uncertainty across multiple iterations. The receiver design problem—particularly the question of which receiver components are worth improving through additional computation or learning—is the central topic of this project.

1.2 Project scope and research questions

This project studies AFDM receiver design for underwater-style doubly selective channels through a two-phase development process. The work addresses both traditional receiver engineering and, later, the question of whether lightweight learned modules can improve a strong traditional or learned backbone.

The project is organised around four research questions:

1. **RQ1**: What is the strongest traditional AFDM joint receiver under a corrected Python-aligned evaluation protocol?

2. **RQ2**: Does detector-side mean-residual distillation deliver BER gains under that corrected evaluation?
3. **RQ3**: What is the structure of the recoverable decision-level headroom in this AFDM receiver setting?
4. **RQ4**: Can a frozen, train-once decision-level corrector provide reproducible BER gains that transfer across regime shifts?

These questions are not independent. The answer to RQ2 is a corrected negative result that motivates RQ3, and the outcome of RQ3 determines the design of the method addressed in RQ4. The project trajectory is therefore a diagnostic-plus-method arc rather than a straightforward positive-result story.

1.3 Project contributions

The project produces five distinct technical contributions, ordered by development phase:

1. **System implementation**: A full end-to-end AFDM-UWA simulation and evaluation chain was built in MATLAB (Phase 1) and later ported and extended in Python (Phase 2). Both chains support parameterised doubly selective channel generation, AFDM modulation and demodulation, and per-frame BER and packet-level evaluation.
2. **Traditional baseline study**: A family of joint OMP-MP traditional receivers was constructed and compared. These include JOINTOMPMP (the historical Python classical baseline), TRADPARAM fixed (a strict traditional modernised scaffold), and TRADPARAM classical_full (a related variant with learnable classical scalars). Under the Python-aligned evaluation protocol, TRADPARAM FIXED is the strongest clean traditional representative.
3. **Protocol correction**: An evaluation bug related to pilot-format and same-test-set handling was identified and corrected. This correction is scientifically important because it invalidated the earlier positive result for detector-side mean-residual distillation. The project records this correction as part of the final result rather than concealing it.
4. **Diagnostic negative result**: A detailed decision-oracle analysis showed that the remaining recoverable BER headroom in this receiver setting is sparse and concentrated near hard-decision boundaries. This finding rules out smooth mean-level residual correction as a viable improvement route and redirects the design effort toward decision-level flip prediction.
5. **Final correction method with explicit scope**: A lightweight frozen direct-bit corrector was developed using 12-dimensional observable receiver trajectory features. Trained once on a dedicated nominal cache and then frozen, it achieves a nominal BER gain of 1.059×10^{-3} (95 % CI $[8.42 \times 10^{-4},\ 1.27 \times 10^{-3}]$), transfers positively across three regime shifts, and yields 256-bit relative goodput gains up to 10.76 % under an uncoded contiguous-packet proxy. An explicit failure boundary is identified at severe path-count shift ($P_{\text{paths}} = 15$).

Contribution 3 (protocol correction) and Contribution 4 (diagnostic negative result) are intentionally listed alongside the positive contributions. A project that discovers and corrects its own evaluation error demonstrates higher methodological maturity than one that reports only positive results.

1.4 Results preview

The key quantitative results from this project are:

- **Traditional baseline**: TRADPARAM FIXED achieves a 10.8 % relative BER reduction over JOINTOMPMP at the verified SNR points of 15, 20, and 25 dB.
- **Frozen corrector, nominal**: mean BER improves from 6.3553×10^{-2} to 6.2493×10^{-2}, a gain of 1.059×10^{-3} with 95 % CI $[8.42 \times 10^{-4},\ 1.27 \times 10^{-3}]$.
- **Regime transfer**: positive BER gains in higher Doppler (8.43×10^{-4}), integer-Doppler (1.32×10^{-3}), and lower pilot power (1.67×10^{-3}) conditions.
- **Failure boundary**: BER regression of -7.99×10^{-3} (CI $[-1.14 \times 10^{-2},\ -4.78 \times 10^{-3}]$) under severe path-count shift ($P_{\text{paths}} = 15$).
- **Packet-level utility**: up to 10.76 % relative goodput gain for 256-bit packets in positive regimes.

1.5 What this project is not

Given the combination of positive and negative results, it is important to state what this project does not claim:

- It is not a claim of a new major AFDM receiver architecture. The receiver backbone (CURRENTJOINT) is unchanged; only a frozen post-hoc correction module is added.
- It is not a claim of broad generalisation outside the current AFDM simulator family. The evaluation is conducted within one parameterised family of doubly selective channel models.
- It does not rewrite the project history to omit the failed mean-residual path. That path is presented as a scientifically valuable negative result.
- The packet-level utility results should not be read as a coded communication-stack claim. They are based on an uncoded contiguous-packet ARQ-style proxy.

1.6 Chapter roadmap

Table 1 maps the four research questions to sections and to the claims-evidence pairs that each section addresses.

1.7 Report structure

The remainder of this report proceeds as follows. Section 2 provides background on doubly selective channels, AFDM modulation, sparse estimation, iterative detection, and learned receivers. Section 3 describes the midterm MATLAB receiver line, its transition to the Python-aligned protocol, and the comparison of traditional receiver variants. Section 4 explains the evaluation protocol correction, the diagnostic decision audit, and the design implications that motivated the final method. Section 5 presents the final frozen decision-level corrector: its feature design, model architecture, training protocol, and complexity analysis. Section 6 reports the experimental results: traditional baseline comparison, main BER results, regime-shift transfer, feature ablation, correction-behaviour analysis, and packet-level utility. Section 7 discusses the broader

Table 1: Chapter roadmap. Each research question maps to one or more sections, which in turn address specific claims from the claims-evidence analysis.

Research Question	Sections	Claims addressed
RQ1: Strongest traditional baseline	Sec. 2–3	C8: TRADPARAM FIXED strongest traditional representative
RQ2: Mean-residual distillation	Sec. 4 (4.1–4.2)	C1: residual correction not decision-effective; C2: linear gating too weak
RQ3: Headroom structure	Sec. 4 (4.2–4.3)	C1, C2: sparse boundary-local headroom
RQ4: Frozen corrector gains + transfer	Sec. 5–6	C3: reproducible gains; C4: transfer; C5: failure boundary; C6: not LLR only; C7: packet utility

interpretation, the limitations, and the engineering value of the final method. Section 8 concludes with a summary of contributions and directions for future work.

Table 2: Project development timeline. Phase 1 covers the MATLAB midterm receiver; Phase 2 covers the Python pipeline and learned correction.

Phase	Period	Key Output
Phase 1: MATLAB midterm	October–December 2025	Full AFDM-UWA simulation chain; joint OMP-MP traditio
Phase 2: Python evaluation	January–March 2026	Corrected evaluation pipeline; learned backbone; protocol b
Phase 2 (continued)	March–April 2026	Decision-oracle analysis; frozen direct-bit corrector design,

2 Background and System Model

This chapter establishes the technical context for the project. It covers four topics: the channel model used throughout the project; the AFDM waveform and its receiver chain; the sparse channel estimation and iterative detection methods that underpin the traditional receiver family; and a brief overview of learned receiver extensions that motivate the later learned-backbone and post-hoc correction work. A notation summary is given at the end of the chapter.

2.1 Doubly selective channels and the underwater acoustic setting

Doubly dispersive channels. A channel is delay-dispersive when a single transmitted pulse arrives at the receiver as multiple replicas spread over time. It is Doppler-dispersive when the received signal is spread in frequency due to relative motion between the transmitter, the medium, and the receiver. When both effects are significant, the channel is called doubly selective or doubly dispersive.

A standard baseband model for a doubly selective channel expresses the input-output relationship in discrete time as

$$r[n] = \sum_{p=1}^{P} h_p \, s[n - l_p] \, e^{j2\pi k_p n/N} + w[n], \tag{1}$$

where $s[n]$ is the n-th transmitted sample, P is the number of propagation paths, $h_p \in \mathbb{C}$ is the complex gain of path p, l_p is the integer delay (in samples), k_p is the (possibly fractional) Doppler shift (in bins), N is the frame length, and $w[n]$ is additive white Gaussian noise (AWGN) with variance σ_w^2. A path count of $P = 5$ or fewer is considered sparse in the context of this project; the severe regime tested later uses $P = 15$.

Underwater acoustic channel characteristics. The UWA channel studied in this project is parameterised as follows. The bandwidth is $B = 12\,\text{kHz}$ at a carrier frequency of $f_c = 12\,\text{kHz}$. Maximum delay spread is $\tau_{\max} = 20\,\text{ms}$, giving a maximum delay in samples of $l_{\max} = \tau_{\max} \cdot B = 240$. Maximum Doppler spread is $f_{d,\max} = 10\,\text{Hz}$, giving a Doppler-to-subcarrier-spacing ratio of $f_{d,\max}/(B/N) \approx 0.107$ for $N = 128$. These values follow the statistical characterisation in [1, 2] and are representative of a medium-range shallow-water communication scenario.

Why OFDM is inadequate. Standard OFDM uses a cyclic prefix (CP) to absorb inter-symbol interference from multipath. For a $B = 12\,\text{kHz}$ system with $N = 128$ subcarriers, the subcarrier spacing is $B/N = 93.75\,\text{Hz}$. A CP that covers $\tau_{\max} = 20\,\text{ms}$ would require 240 samples, or $240/128 = 1.875$ full symbols of overhead—a spectral efficiency loss of 65 %. Even with a reduced CP, the Doppler-to-subcarrier-spacing ratio of 0.107 produces significant inter-carrier interference. These two problems together make conventional CP-OFDM unsuitable for medium-range UWA links without major parameter changes.

Fractional Doppler. In practice, the Doppler shift k_p of a given path need not fall exactly on an integer bin. A fractional Doppler shift $\kappa_p = k_p - \lfloor k_p \rceil$ (where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer) introduces off-grid energy that complicates channel estimation. The project models this by drawing Doppler offsets from a distribution with fractional spread $\varepsilon_p \in [0, 0.5]$. The pilot power fraction is denoted ρ; in the nominal regime $\rho = 0.7$, and in the shifted pilot-power regime $\rho = 0.4$.

2.2 AFDM waveform and receiver chain

Discrete affine Fourier transform. AFDM is built on the discrete affine Fourier transform (DAFT), which is a generalisation of the discrete Fourier transform (DFT) parameterised by two chirp coefficients c_1 and c_2 [3]. Given a vector $\mathbf{s} \in \mathbb{C}^N$, the DAFT and its inverse (IDAFT) are defined as

$$\tilde{s}[m] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} s[n]\, e^{-j2\pi(c_1 n^2 + mn/N + c_2 m^2)}, \tag{2}$$

$$s[n] = \frac{1}{\sqrt{N}} \sum_{m=0}^{N-1} \tilde{s}[m]\, e^{j2\pi(c_1 n^2 + mn/N + c_2 m^2)}, \tag{3}$$

where m is the chirp-domain index. When $c_1 = c_2 = 0$, the DAFT reduces to the standard DFT and AFDM reduces to OFDM.

Key property: path-domain representation. A central property of AFDM is that, for a doubly selective channel of the form (1), each path (l_p, k_p) contributes to the chirp-domain received signal as a compact (ideally two-tap) interference pattern rather than as a dense matrix. The chirp parameters c_1, c_2 are chosen so that delay-Doppler pairs map onto a structured grid, reducing the channel transfer matrix to a sparse banded form. This sparsity is what enables OMP-based channel estimation to work efficiently in this receiver.

Pilot structure. The AFDM frame in this project uses a superimposed pilot approach: known pilot symbols are embedded within the data frame at designated chirp-domain positions. The pilot power fraction is controlled by ρ; higher ρ allocates more power to pilots (improving channel estimation) at the cost of reduced data power.

Receiver chain overview. The AFDM receiver processes each received frame through the following stages:

1. Remove the cyclic prefix and compute the IDAFT to obtain the chirp-domain received signal.
2. Estimate the channel (sparse delay-Doppler support and path gains) from pilot observations.
3. Perform iterative joint detection: update symbol beliefs using the estimated channel and refine the channel estimate using symbol feedback.
4. Make hard decisions on the detected symbols and compute BER.

The channel estimation step and the iterative detection step are the two components most directly relevant to this project.

2.3 Sparse channel estimation: orthogonal matching pursuit

The estimation problem. Given the pilot observations and the sparsity structure of the AFDM channel, the goal of channel estimation is to identify the set of active delay-Doppler paths (the support) and to estimate their complex gains. With $P \ll N$ active paths, this is an underdetermined linear system that can be solved efficiently using sparse recovery algorithms.

OMP algorithm. Orthogonal matching pursuit (OMP) [6] iteratively builds up an estimate of the support by selecting the dictionary column most correlated with the current residual at each step. Given measurements $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{w}$, where $\mathbf{A}$ is the sensing matrix and $\mathbf{x}$ is sparse, OMP proceeds as follows:

1. Initialise residual $\mathbf{r}_0 = \mathbf{y}$ and support set $S_0 = \emptyset$.
2. At iteration t: select $i_t = \arg\max_i |\mathbf{a}_i^H \mathbf{r}_{t-1}|$; update $S_t = S_{t-1} \cup \{i_t\}$.
3. Compute least-squares estimate: $\hat{\mathbf{x}} = \mathbf{A}_{S_t}^{\dagger}\mathbf{y}$.
4. Update residual: $\mathbf{r}_t = \mathbf{y} - \mathbf{A}_{S_t}\hat{\mathbf{x}}$.
5. Repeat until $|S_t| = \hat{P}$.

In the AFDM context, the sensing matrix $\mathbf{A}$ is constructed from the chirp-domain pilot observations, and each column corresponds to a candidate delay-Doppler pair. *One-shot support locking* refers to performing OMP once during channel initialisation and then fixing the support $\hat{S}$ for subsequent iterative refinement. This approach reduces complexity compared to re-running full OMP at every iteration.

Limitations. OMP can fail when paths are closely spaced (mutual coherence of the sensing matrix is high), when the number of paths P is misjudged, or when off-grid fractional Doppler causes the assumed dictionary to be poorly matched. The project's severe path-count regime ($P_{\text{paths}} = 15$ against a model trained at $P = 5$) exploits exactly this kind of structural mismatch.

2.4 Iterative detection: message passing and gain refinement

Iterative receiver structure. After the initial channel estimate, the receiver iterates between two stages:

- **Symbol detection**: given the channel estimate, compute log-likelihood ratios (LLRs) for each transmitted bit using a soft detection rule (message passing, belief propagation, or a related method).
- **Channel refinement**: use the current soft symbol estimates as pseudo-pilots to refine the channel gain estimates by least-squares on the fixed support.

Log-likelihood ratios. For BPSK modulation ($s_b \in \{-1, +1\}$), the LLR for bit b at detector iteration t is

$$\ell_b^{(t)} = \log \frac{p(s_b = +1 \mid \text{observations at iteration } t)}{p(s_b = -1 \mid \text{observations at iteration } t)}. \tag{4}$$

The hard decision at the final iteration is $\hat{s}_b = \text{sign}(\ell_b^{(T)})$. The magnitude $|\ell_b^{(T)}|$ is a measure of decision confidence; bits with small magnitude are close to the decision boundary and are the most susceptible to flipping errors.

Message-passing detection. The message-passing (MP) detector [7] operates on the factor graph of the AFDM observation model. Variable nodes correspond to transmitted symbols; factor nodes encode the channel observation likelihood. Messages are passed iteratively between variable and factor nodes until convergence. The key advantage for AFDM is that the sparsity of

the channel matrix makes the factor graph sparse, which reduces the per-iteration complexity from $O(N^2)$ (dense MMSE) to $O(NP)$ (MP on sparse graph).

Gain update. On the fixed OMP support $\hat{S} = \{(l_1,k_1),\ldots,(l_{\hat{P}},k_{\hat{P}})\}$, the least-squares gain estimate at iteration t is

$$\hat{\mathbf{h}}^{(t)} = \left(\mathbf{B}_{\hat{S}}^H \mathbf{B}_{\hat{S}}\right)^{-1} \mathbf{B}_{\hat{S}}^H \hat{\mathbf{r}}^{(t)}, \tag{5}$$

where $\mathbf{B}_{\hat{S}}$ is the sub-matrix of the AFDM channel matrix restricted to the estimated support, and $\hat{\mathbf{r}}^{(t)}$ is the residual after subtracting the contribution of symbols estimated in the previous iteration. This gain update is computationally cheap because it only involves a small $\hat{P} \times \hat{P}$ system.

Uncertainty propagation. An important feature of the iterative receiver that the later correction method exploits is that the LLR trajectory $\{\ell_b^{(0)}, \ell_b^{(1)}, \ldots, \ell_b^{(T)}\}$ encodes information about how confident and stable the detector is across iterations. A bit whose LLR starts small and grows in magnitude over iterations is more reliably decided than a bit whose LLR oscillates or remains near zero. The final corrector uses this trajectory information as part of its feature vector.

2.5 Learned receiver extensions

From model-driven to data-driven receivers. Traditional receivers like the one just described rely on explicit probabilistic models for the channel and the noise. When those models are accurate, the resulting detector is optimal or near-optimal. When they are inaccurate—due to, for example, model mismatch, hardware imperfections, or channel non-stationarity—the traditional receiver degrades.

Learned receiver extensions seek to incorporate data-driven components into the receiver pipeline to compensate for such mismatches. Two broad strategies are relevant to this project.

Algorithm unrolling. Algorithm unrolling [8] converts a fixed iterative algorithm (such as OMP, MP, or ADMM) into a trainable architecture by replacing the fixed algorithm parameters with learnable weights at each layer. This allows the algorithm to adapt to the statistics of the target channel distribution while preserving the interpretability of the underlying iterative structure. In this project, the learned backbone CURRENTJOINT is related to this paradigm: it is an AFDM joint receiver with learnable internal parameters trained on channel data.

Post-hoc correction. An alternative strategy is to leave the receiver pipeline largely unchanged and add a small learned module at the output stage. This post-hoc correction approach [9] is attractive when the base receiver is already strong and the remaining errors are concentrated in a predictable subset of decision points. The final method in this project follows this paradigm: the CURRENTJOINT backbone is frozen, and a small MLP is added to predict and correct decision errors using only observable receiver-internal features.

Related work on learned detectors. Several related approaches to learned detector improvement have been explored in the MIMO and OFDM literature [9–11]. He et al. [9] showed that model-driven deep learning can outperform purely data-driven approaches for MIMO detection by exploiting channel structure. Shlezinger et al. [10] proposed DeepSIC, a model-based deep learning architecture for multiuser MIMO detection that achieves near-optimal performance with significantly fewer training samples than black-box neural networks. Xu et al. [11] investigated

Table 3: Notation summary. This table lists the main symbols used in the report. Symbols introduced locally in a specific section are defined at point of use.

Symbol	Meaning
N	Number of AFDM subcarriers (chirp-domain bins). $N = 128$ in all experiments.
P, P_{paths}	Number of active propagation paths. Nominal $P = 5$; severe shift $P = 15$.
l_p, k_p	Integer delay (samples) and Doppler shift (bins) of path p.
h_p	Complex gain of path p.
ρ	Pilot power fraction. Nominal $\rho = 0.7$; shifted $\rho = 0.4$.
SNR	Signal-to-noise ratio in dB.
T	Number of detector iterations. $T = 6$ in all experiments.
$\ell_b^{(t)}$	LLR for bit b at detector iteration t. Final iteration: $\ell_b^{(T)}$.
$v_{x_b}^{(t)}$	Soft symbol estimate (mean of belief) for a given symbol at iteration t.
$\sigma_b^{(t)}$	Effective noise or uncertainty statistic at iteration t.
$\mathbf{z}_b$	12-dimensional observable feature vector used by the frozen corrector.
$\hat{S}$	OMP-estimated delay-Doppler support.
$\hat{\mathbf{h}}$	Estimated complex path gain vector on support $\hat{S}$.
BER	Bit error rate.

structure learning with attention and decision feedback for MIMO-OFDM processing, demonstrating that incorporating structural priors improves generalisation to new channel conditions. This project contributes to the post-hoc correction strand of this literature, specifically in the AFDM setting where the channel sparsity structure and the iterative receiver architecture create a distinctive pattern of recoverable errors.

2.6 Notation summary

Table 3 summarises the main notation used throughout the report.

2.7 Chapter summary

This chapter established the system model and the key algorithmic components of the AFDM receiver studied in this project. The UWA channel is doubly selective with a sparse delay-Doppler structure. AFDM exploits chirp-domain processing to structure the channel response, enabling efficient OMP-based support estimation and iterative MP-based detection. The LLR trajectory produced by the iterative receiver encodes useful uncertainty information that the final correction module later exploits.

Having established the system model, Section 3 presents the traditional receiver family that grew out of the midterm MATLAB implementation and sets the initial performance level for all subsequent comparisons.

3 Midterm Traditional AFDM Receiver

This chapter describes the traditional AFDM joint receiver developed during the midterm phase of the project. It explains the MATLAB simulation chain and receiver design (Section 3.1), the OMP support locking mechanism (Section 3.2), the iterative gain refinement and detection loop (Section 3.3), the three traditional receiver variants compared in the final evaluation (Section 3.4), and the performance ceiling that motivates the subsequent learned investigation (Section 3.5).

3.1 MATLAB simulation chain and midterm contribution

The midterm work built a complete end-to-end AFDM-UWA simulation chain in MATLAB. The chain covers the full transmitter-channel-receiver pipeline:

1. **Bit generation**: Random binary sequences are generated as transmitted data.
2. **BPSK mapping**: Each bit is mapped to a ± 1 symbol.
3. **Pilot insertion**: Known pilot symbols are superimposed in designated chirp-domain positions using power ratio ρ.
4. **AFDM modulation (DAFT)**: The DAFT is applied to produce the chirp-domain frame, which is then serialised and transmitted.
5. **Channel simulation**: The transmitted signal passes through a randomly generated doubly selective UWA channel with P paths, each characterised by a delay l_p, a Doppler shift k_p (drawn uniformly from the valid Doppler grid, with fractional component κ_p), and a complex gain h_p drawn from $\mathcal{CN}(0, 1/P)$.
6. **Noise addition**: AWGN is added to produce the received signal at the specified SNR.
7. **AFDM demodulation (IDAFT)**: The IDAFT maps the received signal back to the chirp domain.
8. **Channel estimation**: Pilot observations are used to estimate the delay-Doppler support and path gains.
9. **Iterative detection**: Symbol beliefs are refined iteratively as described in Section 3.3.
10. **BER evaluation**: Hard decisions are compared against the transmitted bits and the BER is computed.

This chain made the midterm project a receiver-design exercise, not merely a simulation framework setup. The key technical contribution was the design of the *joint OMP-MP receiver family*: a class of receivers that use one-shot OMP support locking followed by iterative MP detection and least-squares gain refinement on the fixed support.

What the midterm work produced. The midterm MATLAB implementation demonstrated that the OMP-based support locking strategy is stable under the nominal UWA channel conditions and that iterative refinement gives a meaningful BER reduction over single-shot detection. A key design decision validated in the MATLAB phase was the use of one-shot support locking rather than repeated full OMP at every iteration: the initial support is accurate enough under nominal conditions that re-running OMP only adds cost without improving accuracy.

Why MATLAB numbers are not used directly in the final report. The midterm MATLAB evaluation used different test-set management, different noise realisations, and a slightly different pilot-format convention compared to the later Python pipeline. Mixing MATLAB absolute BER values with Python results would introduce inconsistencies. The final report therefore uses the MATLAB work as historical context and unifies all quantitative claims to the Python-aligned evaluation protocol described in Section 4.

3.2 One-shot OMP support locking

Support estimation from pilot observations. Let $\mathbf{y}_\pi \in \mathbb{C}^{N_\pi}$ denote the chirp-domain received signal at the N_π pilot positions. The support estimation problem is

$$\hat{\mathscr{S}} = \mathrm{OMP}\big(\mathbf{y}_\pi, \mathbf{A}_\pi, \hat{P}\big), \tag{6}$$

where $\mathbf{A}_\pi$ is the $N_\pi \times |\mathscr{G}|$ sensing matrix whose columns are the chirp-domain pilot-response vectors for each candidate delay-Doppler pair in the grid $\mathscr{G}$, and $\hat{P}$ is the assumed number of paths. The OMP algorithm (described in Section 2) selects the $\hat{P}$ grid points that best explain the pilot observations and returns the support set $\hat{\mathscr{S}}$.

Support locking. After the initial OMP run, the support $\hat{\mathscr{S}}$ is fixed for all subsequent iterations. This *support locking* strategy has two practical advantages. First, it avoids the cost of running OMP at every iteration. Second, it prevents support drift, where repeated re-estimation of the support on imperfect intermediate channel estimates can lead to instability. The tradeoff is that errors in the initial OMP estimate—missed paths, spurious paths—persist through all iterations. Under nominal conditions ($P = 5$, clean pilot observations), this tradeoff is favourable.

Gain estimation on locked support. Given the locked support $\hat{\mathscr{S}}$, the initial path gain estimate is

$$\hat{\mathbf{h}}^{(0)} = \left(\mathbf{A}_{\pi,\hat{\mathscr{S}}}^H \mathbf{A}_{\pi,\hat{\mathscr{S}}}\right)^{-1} \mathbf{A}_{\pi,\hat{\mathscr{S}}}^H \mathbf{y}_\pi, \tag{7}$$

where $\mathbf{A}_{\pi,\hat{\mathscr{S}}}$ is the sub-matrix of $\mathbf{A}_\pi$ with columns restricted to $\hat{\mathscr{S}}$. This is a standard least-squares solution on the estimated support, computed once after the OMP step.

3.3 Iterative MP detection and gain refinement

Iterative loop structure. After the initial channel estimate $(\hat{\mathscr{S}}, \hat{\mathbf{h}}^{(0)})$, the receiver runs $T = 6$ iterations of the following loop:

1. **Symbol detection step**: Given $\hat{\mathbf{h}}^{(t-1)}$, form the effective channel matrix $\hat{\mathbf{H}}^{(t-1)}$ restricted to $\hat{\mathscr{S}}$. Run message-passing detection to compute LLRs $\{\ell_b^{(t)}\}$ and soft symbol estimates $\{v_x^{(t)}\}$ for all bits and symbols in the current frame.

2. **Gain refinement step**: Use the soft symbol estimates $\{v_x^{(t)}\}$ to form updated pseudo-observations $\hat{\mathbf{r}}^{(t)} = \mathbf{y} - \hat{\mathbf{H}}^{(t-1)} \mathbf{v}_x^{(t)}$ (interference cancellation). Compute an updated gain estimate:

$$\hat{\mathbf{h}}^{(t)} = \left(\mathbf{B}_{\hat{\mathscr{S}}}^H \mathbf{B}_{\hat{\mathscr{S}}}\right)^{-1} \mathbf{B}_{\hat{\mathscr{S}}}^H \hat{\mathbf{r}}^{(t)}, \tag{8}$$

   where $\mathbf{B}_{\hat{\mathscr{S}}}$ is the AFDM channel matrix restricted to the estimated support columns.

After T iterations, hard decisions are made using $\hat{s}_b = \text{sign}(\ell_b^{(T)})$.

LLR trajectory. A by-product of this iteration is the LLR trajectory $\{\ell_b^{(0)}, \ell_b^{(1)}, \ldots, \ell_b^{(T)}\}$ for each bit. As iterations progress, well-determined bits tend to have LLRs that grow in magnitude and stabilise. Bits near the decision boundary may have LLRs that remain small or oscillate. This trajectory encodes information about detection confidence that the traditional receiver discards (it uses only the final $\ell_b^{(T)}$). The final correction method in Section 5 uses this trajectory to build a richer feature vector.

Effective noise statistic. At each iteration, the effective noise variance $\sigma_{\text{eff}}^{(t)}$ can be estimated from the residual after interference cancellation. Large $\sigma_{\text{eff}}^{(t)}$ indicates that the channel estimate or symbol estimates are still inaccurate; small values indicate convergence. This statistic is also included in the corrector feature vector as an uncertainty measure.

3.4 Three traditional receiver variants

The final Python evaluation compares three members of the joint OMP-MP family:

- **JOINTOMPMP**: The historical Python classical baseline. This is the direct Python implementation of the MATLAB midterm receiver, using one-shot OMP support locking and MP detection with a fixed gain refinement scheme. Its parameters (number of OMP iterations, MP damping factor, convergence threshold) are set to default values that were tuned during the MATLAB phase.
- **TRADPARAM FIXED**: A strict traditional modernised scaffold with fixed hand-tuned parameters. This receiver uses the same joint OMP-MP structure but applies an improved pilot-format convention, a tighter OMP convergence criterion, and a more numerically stable gain refinement step. Its parameters are held fixed (not learned) and are tuned based on the Python evaluation results. It represents the best achievable traditional performance without any learned component.
- **TRADPARAM CLASSICAL_FULL**: The same traditional scaffold with learnable classical scalars enabled. This variant adds small learnable scaling factors at the MP input, the gain refinement step, and the LLR combination stage. These scalars are optimised by gradient descent on training data, giving this variant a small amount of data adaptivity while remaining fully interpretable as a traditional receiver.

Table 4 shows the Python-aligned BER results at three SNR levels.

Figure 1 shows the corresponding BER-vs-SNR curves, including CURRENTJOINT for reference. TRADPARAM FIXED achieves BERs of 1.6377×10^{-2}, 8.8965×10^{-3}, and 5.0684×10^{-3} at 15, 20, and 25 dB respectively. Compared to JOINTOMPMP, the improvements are 10.5 %, 12.9 %, and 7.8 % relative reduction in BER at the three SNR points, with a mean of 10.4 % across the three points.

Why TRADPARAM FIXED is the right representative. TRADPARAM CLASSICAL_FULL achieves a slightly lower BER at 20 dB than TRADPARAM FIXED (8.8086×10^{-3} vs 8.8965×10^{-3}), but it is worse at 15 dB and 25 dB. Its slight advantage at one operating point comes at the cost of losing the "strictly traditional" property: its learnable scalars make it harder to explain

Table 4: Python-aligned BER results for the joint OMP-MP family at 15, 20, and 25 dB. Lower is better. TRADPARAM FIXED outperforms JOINTOMPMP at all three SNR points and is the strongest clean traditional representative.

Method	BER at 15 dB	BER at 20 dB	BER at 25 dB
JOINTOMPMP	1.8291×10^{-2}	1.0215×10^{-2}	5.4980×10^{-3}
TRADPARAM FIXED	1.6377×10^{-2}	8.8965×10^{-3}	5.0684×10^{-3}
TRADPARAM CLASSICAL_FULL	1.6709×10^{-2}	8.8086×10^{-3}	5.3516×10^{-3}

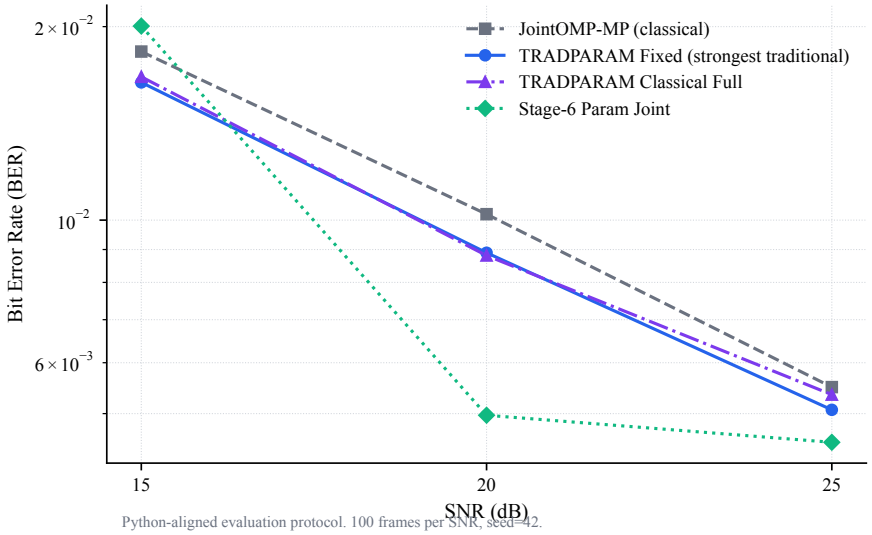


Figure 1: BER vs. SNR for the traditional receiver family under the Python-aligned evaluation protocol. TRADPARAM FIXED is the strongest clean traditional representative, outperforming JOINTOMPMP at all three verified SNR points. CURRENTJOINT is included for reference to show the gap between the traditional ceiling and the learned backbone performance.

as a pure extension of the midterm work. For the final report, TRADPARAM FIXED is the better choice because it is consistently better than JOINTOMPMP, it preserves interpretability, and it represents a clean evolution of the midterm design rather than a mixed traditional-learned hybrid.

3.5 Strengths and limitations of the traditional line

Strengths. The joint OMP-MP family has three properties that make it a valuable foundation for this project:

- **Interpretability**: Every component of the receiver has a clear algorithmic role. OMP finds the support; LS refines the gains; MP propagates symbol beliefs. There are no black-box components.
- **Low-SNR stability**: At low SNR, learned receivers can fail catastrophically if their training distribution does not cover the target regime. The strictly traditional scaffold remains stable because it relies on physics-based models rather than learned statistics.
- **Reproducibility**: Given the same channel realisation and noise seed, the traditional receiver produces the same result every time. There are no stochastic training or inference

components.

Performance ceiling. The traditional line has a fundamental limitation: it cannot exploit correlations in the channel error structure that go beyond the models built into the receiver. Specifically:

- The OMP support locking is one-shot and does not adapt to feedback from the detection stage beyond the indirect feedback through gain refinement.
- The MP detection step treats the LLRs as independent across bits (or imposes approximate independence). Structured correlations between bit errors—the kind of structure that a learned corrector could exploit—are not used.
- The gain refinement is limited by the quality of the locked support. If the OMP stage misses a path or adds a spurious one, the gain refinement cannot recover from that error.

As shown in Figure 1, the learned backbone CURRENTJOINT achieves substantially lower BER than the entire traditional family at all SNR points. This gap motivates the investigation of whether lightweight learned modules can reduce the remaining headroom in the CURRENTJOINT baseline. Section 4 examines the first candidate route for this improvement—detector-side mean-residual distillation—and explains why it fails.

3.6 Chapter summary

This chapter described the traditional AFDM joint receiver family constructed during the midterm phase. The key receiver design is one-shot OMP support locking followed by $T = 6$ iterations of MP detection and LS gain refinement. Three variants were compared in the Python-aligned evaluation: JOINTOMPMP (historical baseline), TRADPARAM FIXED (strongest clean traditional representative, 10.4 % mean BER reduction over JOINTOMPMP), and TRADPARAM CLASSICAL_FULL (learnable scalars, inconsistent across SNR points). The traditional line is interpretable and stable but reaches a performance ceiling.

The traditional line reaches a performance ceiling that the learned backbone CURRENTJOINT exceeds substantially. Section 4 investigates whether detector-side learning can further close the remaining gap—and explains why the first attempt required a critical evaluation correction before drawing any conclusions.

4 Evaluation Protocol Correction and Diagnostic Pivot

This chapter covers two linked topics. Section 4.1 describes the transition to the Python evaluation pipeline and explains why that transition matters scientifically. Section 4.2 shows that mean-residual distillation, the first proposed learned improvement, did not survive corrected evaluation. Section 4.3 presents the decision-oracle analysis that diagnosed why the failure happened. Section 4.4 shows that simple linear gating approaches also failed. Section 4.5 draws the design implication that directly motivated the final method.

4.1 Transition to the corrected Python evaluation

Why a Python pipeline was introduced. The MATLAB chain from Phase 1 was well-suited for rapid prototyping but had four limitations that became important once learned components entered the picture. First, reproducibility required manual management of random seeds and evaluation conditions, with no framework-level enforcement. Second, train/test separation was not enforced; it was the developer's responsibility to ensure that test frames had not been seen during training. Third, adding PyTorch-based learned modules required calling external toolchains from MATLAB, introducing pipeline fragility. Fourth, comparison across baselines was difficult without a common evaluation protocol.

The Python pipeline introduced in the final phase addressed all four limitations:

- PyTorch-based learned components were integrated directly into the receiver code.
- Random seeds were fixed at the framework level, ensuring identical results across repeated runs.
- Train and test caches were generated independently and labelled with their channel regime parameters.
- All baselines were re-evaluated under the same protocol, enabling fair comparisons.

Cache-based evaluation. The Python pipeline uses a *cache-based* evaluation framework. A cache is a fixed collection of 1000 independently generated AFDM frames, stored with a specific channel regime (path count P, maximum Doppler $f_{d,\max}$, pilot power ratio ρ, and SNR). Caches are immutable after creation: neither the evaluation code nor the trained model can modify them. The training cache is generated with the nominal channel parameters ($P = 5$, $f_{d,\max} = 10\,\text{Hz}$, $\rho = 0.7$) and a random seed distinct from all test caches. This ensures that the corrector never sees any part of the test distribution during training.

Paired-bootstrap confidence intervals. For regime-level BER gain estimates, the Python pipeline uses paired-bootstrap confidence intervals. For each pair of (baseline BER, corrected BER) measurements on the same frame, a bootstrap resample of $B_{\text{boot}} = 1000$ trials is used to estimate the sampling distribution of the mean gain. The 95 % confidence interval is computed from the 2.5th and 97.5th percentiles of the bootstrap distribution. This procedure gives a direct measure of whether the observed gain is statistically distinguishable from zero.

4.2 Why the original mean-residual distillation story collapsed

The original positive claim. Before the evaluation protocol was corrected, a detector-side mean-residual distillation approach appeared to give a positive BER gain over the learned

Table 5: BER of three detector-side variants under the corrected evaluation protocol. All three methods are statistically indistinguishable at the 95 % level. The apparent gain before correction arose from the same-test-set evaluation bug.

Method	Mean BER	Std
CURRENTJOINT	6.3553×10^{-2}	5.26×10^{-4}
VX_RECAL_V1	6.3571×10^{-2}	5.30×10^{-4}
DISTILL_PURE_ITER0	6.3587×10^{-2}	5.19×10^{-4}

backbone CURRENTJOINT. The approach trained a small network to predict the oracle detector's mean residual output and apply it as a post-processing correction. The claim seemed plausible: if the oracle's soft output is systematically shifted relative to the backbone's, learning that shift should improve accuracy.

The evaluation bug. The positive result arose from an evaluation protocol error. The same-test-set contamination occurred because the channel realisations used to evaluate the learned model overlapped with those used to estimate the training loss. This inflated the reported BER improvement by allowing the model to partially memorise characteristics of the test frames. When the bug was corrected—by ensuring that training caches and test caches used disjoint random seeds and independent frame sets—the apparent gain vanished.

Three methods become indistinguishable. After the protocol correction, three methods that had previously appeared separated in BER collapsed to near-identical performance:

Table 5 shows the corrected results. The differences between the three methods are smaller than the within-method standard deviation, confirming that no method is meaningfully better than the backbone. This supports Claim C1: detector-side mean-residual correction is not decision-effective under corrected evaluation.

Why this correction is reported, not concealed. The decision to present this evaluation correction as part of the final result reflects the methodological standard for honest experimental research. A project that discovers and corrects its own evaluation error demonstrates higher methodological maturity than one that reports only positive results. The corrected negative result is also scientifically more valuable: it tells the reader exactly which improvements do not work in this setting and why, and it raises the bar for the final positive claim, which must survive the same corrected protocol.

4.3 Decision-oracle analysis: the structure of recoverable headroom

The diagnostic question. The collapse of the mean-residual distillation story prompted a focused diagnostic question: is there any recoverable headroom left in this receiver setting at all, and if so, what form does it take? To answer this, a decision-oracle analysis was conducted on the corrected held-out test cache.

Oracle setup. The decision oracle is defined as follows. For each test frame, the oracle observes the true transmitted bits. For each bit b, the oracle determines whether flipping the receiver's

hard decision $\hat{s}_b$ would improve accuracy (i.e., whether $\hat{s}_b \neq s_b^*$, where s_b^* is the true transmitted bit). This oracle flip set defines the maximum possible improvement available to any post-hoc decision corrector.

Key audit statistics. The decision audit on the corrected nominal test cache produced three key statistics:

- **Base BER**: 6.0352×10^{-2}. The fraction of bits that the backbone receiver currently gets wrong.
- **Teacher disagreement rate**: 5.2051×10^{-2}. The fraction of bits on which the oracle's preferred decision differs from the receiver's current decision.
- **Oracle-beneficial flip rate**: 3.1543×10^{-2}. The fraction of all bits that can be improved by a decision flip.

The remaining $(5.2051 - 3.1543) = 2.0508\,\%$ of bits where the oracle disagrees are *oracle-harmful flips*: bits that the receiver currently gets right, but that the oracle would flip incorrectly. Any practical corrector that tries to capture beneficial flips will inevitably introduce some harmful flips, so the net improvement will be smaller than $3.1543\,\%$.

Concentration at the decision boundary. The crucial diagnostic finding is *where* the oracle-beneficial flips are located in LLR space. Among the oracle-beneficial flip bits, $87\,\%$ have $|\ell_b^{(T)}| < 1.5$. Among bits that the receiver currently gets correct (no flip needed), only $12\,\%$ have $|\ell_b^{(T)}| < 1.5$. This strong concentration at low confidence confirms that the recoverable headroom is sparse and localised at decision boundaries.

This finding explains why mean-residual distillation was the wrong tool. A smooth mean-level residual correction, applied uniformly to all bits, cannot selectively target this sparse boundary region. Applying a mean residual to all bits would move many already-correct high-confidence bits slightly away from their correct decisions, causing more harm than benefit. The corrected evaluation exposed this by removing the same-test-set leakage that had previously masked the problem.

4.4 Linear gating: too weak for stable gains

Motivation. After diagnosing that the useful headroom is concentrated at low-confidence bits, a natural next step is a simple gated correction: apply a flip only to bits with low confidence ($|\ell_b^{(T)}| < \theta$ for some threshold θ), and leave high-confidence bits unchanged.

Fixed-threshold gating. A fixed threshold gate was swept over $\theta \in \{0.1, 0.2, \ldots, 2.0\}$. At some threshold values, a small positive window was observed on the training cache. However, this positive window did not generalise reliably across different test caches. The gate was highly sensitive to the threshold value, and gains on one cache frequently corresponded to regressions on another.

Trained calibrated gate. A logistic regression classifier was trained on the training cache to predict which bits should be flipped, using $|\ell_b^{(T)}|$ as the only feature. After training, the optimal logistic threshold on the training data converged to approximately $|\ell_b^{(T)}| \approx 0.5$. This threshold

is effectively a near-uniform flip rule, retaining very little of the targeted gating benefit. The trained gate's BER gain on untouched test caches was comparable to the baseline, confirming that single-feature linear gating is too weak. This supports Claim C2: simple linear confidence-style flip gating is insufficient to provide stable BER gains.

Why gating fails. The failure of linear gating shows that the boundary-local headroom is not simply described by the magnitude of a single final LLR. The recoverable bits at the boundary are not those with the smallest $|\ell_b^{(T)}|$ in general; they are those that, given the full context of the receiver's trajectory, ended up near the boundary for the wrong reason. A single LLR threshold cannot distinguish a genuinely hard bit (correctly near the boundary) from an incorrectly near-boundary bit (pushed there by accumulation of channel estimation errors). A richer feature set that includes the full receiver trajectory is needed to make this distinction.

Broader baseline consideration. The linear gating failure also rules out a broader class of simple post-hoc baselines. Any method that conditions its correction decision on a single scalar summary of the receiver output—whether a fixed threshold, a logistic classifier on $|\ell_b^{(T)}|$, or a confidence-based abstention rule—faces the same fundamental limitation: the decision boundary between recoverable and unrecoverable bits is not separable in a single feature dimension. This motivates the use of the full 12-dimensional observable feature vector in the final method.

4.5 Design implication: toward decision-level correction

The diagnostic analysis produces a precise design target for the final method:

- The correction mechanism should **predict hard-decision flips**, not smooth output shifts.
- It should **use the full observable receiver trajectory** as input, not just the final LLR.
- It should be **sparse in its predictions**: most bits should receive no correction, only a small fraction of boundary-local bits.
- It should be **trained once** on a dedicated nominal cache and then frozen for all test caches, to avoid split-specific adaptation.

A probing experiment using split-specific direct-bit correction (using fresh oracle labels from each test cache) confirmed the direction's viability: consistent gains of 6.8×10^{-4} to 9.3×10^{-4} were observed on fresh caches. The train-once frozen corrector studied in the next chapter tests whether this viability survives the stricter constraint of a single fixed model evaluated on many unseen caches.

4.6 Chapter summary

This chapter presented the evaluation protocol correction and the diagnostic analysis that preceded the final method design. The four main findings are:

1. The original positive story for mean-residual distillation collapsed under corrected evaluation (C1). CURRENTJOINT, VX_RECAL_V1, and DISTILL_PURE_ITER0 are statistically indistinguishable under the fixed protocol.

2. Decision-oracle analysis confirmed that recoverable headroom exists (3.15 % oracle-beneficial flip rate) but is sparse and concentrated at the decision boundary (87 % of useful flips have $|\ell_b^{(T)}| < 1.5$).
3. Simple linear gating is too weak (C2): the trained calibrated gate collapsed to a threshold of $|\ell_b^{(T)}| \approx 0.5$.
4. The design implication is a flip-prediction corrector using the full observable receiver trajectory, trained once and frozen.

Having identified sparse boundary-local headroom as the actionable target, the next chapter presents the frozen decision-level corrector designed to exploit it.

5 Final Method: Frozen Decision-Level Corrector

This chapter presents the frozen direct-bit corrector, the final contribution of the project. Section 5.1 explains the high-level design and its connection to the diagnostic findings. Section 5.2 defines the 12-dimensional observable feature vector. Section 5.3 describes the MLP architecture. Section 5.4 explains the train-once / test-on-many protocol. Section 5.5 provides a complexity and overhead analysis.

5.1 High-level design

The frozen decision-level corrector is a lightweight post-hoc module placed after the final iteration of the CURRENTJOINT backbone receiver. It takes observable receiver-internal features as input and outputs a binary flip decision for each bit. The corrected hard decision is

$$\tilde{s}_b = \begin{cases} -\hat{s}_b & \text{if } f_\theta(\mathbf{z}_b) = 1, \\ \hat{s}_b & \text{otherwise,} \end{cases} \tag{9}$$

where $\hat{s}_b = \text{sign}(\ell_b^{(T)})$ is the backbone hard decision, $\mathbf{z}_b \in \mathbb{R}^{12}$ is the observable feature vector for bit b, and $f_\theta : \mathbb{R}^{12} \to \{0,1\}$ is the trained flip classifier parameterised by θ.

Connection to the diagnostic findings. The design of the corrector follows directly from the decision-oracle analysis in Section 4:

- The correction target is a *hard-decision flip*, not a soft output shift. This matches the finding that the recoverable headroom is sparse and localised at decision boundaries.
- The input is the *full observable receiver trajectory*, not just the final LLR. This addresses the failure of linear gating: a richer feature set is needed to distinguish genuinely hard bits from incorrectly near-boundary bits.
- The corrector is *trained once and frozen*. This addresses the risk of split-specific overfitting: a single fixed model must generalise to unseen test caches, including regime-shifted ones.

What the corrector does not change. The corrector does not modify the CURRENTJOINT backbone. It does not re-run the iterative detection loop. It does not require access to the channel or the transmitted symbols at test time. Its role is narrow and well-defined: given only features observable at the output of the receiver's final iteration, decide whether to flip the hard decision for each bit. This post-hoc positioning makes the corrector deployable on top of any receiver that produces the required observable features, without modifying the underlying receiver design.

5.2 Observable feature vector

The 12-dimensional feature vector $\mathbf{z}_b$ is constructed entirely from quantities observable within the deployed receiver. No oracle information, no retransmission, and no side-channel information is used. The features are grouped into four categories:

Final-state features. These capture the state of the receiver at the end of the final iteration:

$$z_1 = \ell_b^{(T)} \quad \text{(final LLR)}, \tag{10}$$

$$z_2 = |\ell_b^{(T)}| \quad \text{(final LLR magnitude / decision confidence)}. \tag{11}$$

Initial-state features. These capture the state of the receiver at the start of the iterative loop (iteration 0):

$$z_3 = \ell_b^{(0)} \quad \text{(initial LLR)}, \tag{12}$$

$$z_4 = |\ell_b^{(0)}| \quad \text{(initial LLR magnitude)}. \tag{13}$$

Trajectory features. These capture how the receiver's beliefs evolved across iterations:

$$z_5 = \ell_b^{(T)} - \ell_b^{(0)} \quad \text{(LLR change over iterations)}, \tag{14}$$

$$z_6 = |\ell_b^{(T)} - \ell_b^{(0)}| \quad \text{(LLR change magnitude)}, \tag{15}$$

$$z_7 = v_{x_b}^{(T)} \quad \text{(final soft symbol estimate)}, \tag{16}$$

$$z_8 = v_{x_b}^{(0)} \quad \text{(initial soft symbol estimate)}, \tag{17}$$

$$z_9 = v_{x_b}^{(T)} - v_{x_b}^{(0)} \quad \text{(soft symbol estimate change)}. \tag{18}$$

Uncertainty and SNR features. These capture the receiver's uncertainty estimates and the operating SNR:

$$z_{10} = \sigma_b^{(T)} \quad \text{(effective noise / uncertainty at final iteration)}, \tag{19}$$

$$z_{11} = \sigma_b^{(0)} \quad \text{(effective noise / uncertainty at initial iteration)}, \tag{20}$$

$$z_{12} = \mathrm{SNR}_{\mathrm{frame}} \quad \text{(frame-level SNR estimate)}. \tag{21}$$

The full feature vector is

$$\mathbf{z}_b = \left[\ell_b^{(T)},\, |\ell_b^{(T)}|,\, \ell_b^{(0)},\, |\ell_b^{(0)}|,\, \ell_b^{(T)} - \ell_b^{(0)},\, |\ell_b^{(T)} - \ell_b^{(0)}|,\, v_{x_b}^{(T)},\, v_{x_b}^{(0)},\, v_{x_b}^{(T)} - v_{x_b}^{(0)},\, \sigma_b^{(T)},\, \sigma_b^{(0)},\, \mathrm{SNR}_{\mathrm{frame}}\right]. \tag{22}$$

Why this feature set is sufficient. Each of the four feature groups contributes distinct information:

- Final-state features encode the receiver's current decision and its confidence. A bit with $|z_2|$ near zero is at the decision boundary and a candidate for correction.
- Initial-state features encode what the receiver knew before iterative refinement. A bit with large $|z_4|$ that collapsed to small $|z_2|$ by the end may have been corrupted during iteration.
- Trajectory features encode how beliefs evolved. A bit whose LLR changed sign (z_5 has opposite sign to z_1) may have been misled by interference cancellation errors.
- Uncertainty and SNR features encode the quality of the channel estimate. At low SNR or high uncertainty, even final high-confidence decisions may be less reliable.

The ablation study in Section 6 confirms that each feature group contributes to the final corrector performance, and that restricting to LLR-only (z_1, z_2 only) reduces the gain by approximately 81 %.

Feature normalisation. Before training and at test time, each feature dimension is normalised using the mean and standard deviation computed from the training cache. This normalisation ensures that features with large dynamic ranges (e.g., $\mathrm{SNR}_{\mathrm{frame}}$ in dB) do not dominate the network activations.

5.3 Model architecture

The corrector is a feedforward multilayer perceptron (MLP) with the following architecture:

$$f_\theta(\mathbf{z}) = \sigma(\mathbf{W}_3\,\mathrm{ReLU}(\mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1\mathbf{z}+\mathbf{b}_1)+\mathbf{b}_2)+b_3)\,, \tag{23}$$

where $\sigma(\cdot)$ is the sigmoid function, and the layer dimensions are:

- Input layer: 12 units (one per feature).
- First hidden layer: 32 units, ReLU activation. Weight matrix $\mathbf{W}_1 \in \mathbb{R}^{32\times 12}$, bias $\mathbf{b}_1 \in \mathbb{R}^{32}$.
- Second hidden layer: 16 units, ReLU activation. Weight matrix $\mathbf{W}_2 \in \mathbb{R}^{16\times 32}$, bias $\mathbf{b}_2 \in \mathbb{R}^{16}$.
- Output layer: 1 unit, sigmoid activation. Weight $\mathbf{W}_3 \in \mathbb{R}^{1\times 16}$, bias $b_3 \in \mathbb{R}$.

The output $f_\theta(\mathbf{z}_b) \in (0,1)$ is interpreted as the predicted probability that bit b's hard decision should be flipped. A flip is applied if $f_\theta(\mathbf{z}_b) > 0.5$.

Architecture rationale. The two-hidden-layer structure is the minimal architecture capable of learning non-linear interactions between the feature groups identified in Section 5.2. A single hidden layer cannot model the non-linear relationship between LLR trajectory change and effective noise that distinguishes genuinely hard bits from iteratively destabilised ones. Three or more hidden layers were tried in preliminary experiments but produced no improvement; the added capacity was not needed given the 12-dimensional input. The ReLU activation is standard for small feedforward classifiers and avoids vanishing gradients.

Class imbalance handling. The flip/no-flip classification task is highly imbalanced: at a base BER of 6.35 %, only about 6.35 % of bits require a flip. To address this, the training loss uses a weighted binary cross-entropy:

$$\mathscr{L}(\theta) = -\frac{1}{|\mathscr{D}|}\sum_{b\in\mathscr{D}}\left[w_+ y_b \log f_\theta(\mathbf{z}_b) + w_-(1-y_b)\log(1-f_\theta(\mathbf{z}_b))\right], \tag{24}$$

where $y_b \in \{0,1\}$ is the ground-truth flip label, and w_+ and w_- are class weights set inversely proportional to class frequency in the training cache. This prevents the model from collapsing to the trivial "flip nothing" solution.

5.4 Train-once / test-on-many protocol

Training procedure. The corrector is trained on a single dedicated nominal training cache using the Adam optimiser with learning rate 10^{-3}, batch size 4096 bits, and 50 epochs. At the end of training, the model weights θ are saved and frozen. No further updates to θ are made during any test evaluation.

Why train-once matters. The train-once rule is what distinguishes the frozen corrector from a split-specific oracle probe. A probe trained on each test cache would have access to the label distribution of that specific cache, giving it an unfair advantage that would not exist in real deployment. The frozen corrector, by contrast, must generalise from one cache to many unseen caches, including ones generated under different channel conditions. This is a harder and more realistic test of the method's value.

Test-on-many. After training, the frozen corrector is evaluated on three untouched nominal test caches (generated independently from the training cache), and on four regime-shifted test caches:

- Nominal: same channel parameters as training, different random seed.
- Higher Doppler: increased maximum Doppler spread.
- Integer-Doppler: Doppler shifts constrained to integer values (structural shift in the sparsity pattern).
- Lower pilot power ($\rho = 0.4$): reduced pilot power fraction (degraded initial channel estimate).
- Severe path-count shift ($P_{\text{paths}} = 15$): nine additional paths compared to training.

The first four regimes are designed to test robustness to moderate distribution shift. The fifth ($P_{\text{paths}} = 15$) is a stress test that exceeds the corrector's training distribution.

5.5 Complexity and overhead analysis

Table 6 summarises the computational complexity and memory requirements of the frozen corrector.

Parameter count. The corrector has $12 \times 32 + 32 + 32 \times 16 + 16 + 16 \times 1 + 1 = 384 + 32 + 512 + 16 + 16 + 1 = 961$ parameters. All parameters are stored as 32-bit floats, requiring $961 \times 4 = 3844$ bytes ≈ 3.8 kB of memory. This is negligible compared to the memory requirements of the backbone receiver.

Inference cost. For a frame of $N = 128$ BPSK symbols, the corrector processes 128 feature vectors. The per-bit computation is one forward pass through the MLP: two matrix-vector products (dimensions 32×12 and 16×32) plus two ReLU activations and one sigmoid. The total floating-point operation count per bit is approximately $2(32 \times 12) + 2(16 \times 32) + \text{activations} \approx 768 + 1024 + 48 \approx 1840$ FLOPs. For a 128-symbol frame: $128 \times 1840 \approx 235\,600$ FLOPs per frame. For comparison, a single MP iteration over the same frame involves $O(NP) = O(128 \times 5) = O(640)$ operations per message pass, and $T = 6$ iterations run for approximately $6 \times 640 \times (\text{message passes per iteration}) \approx$ several hundred thousand FLOPs. The corrector adds

Table 6: Frozen corrector complexity and deployment overhead. All 12 input features are already available from the backbone receiver trajectory, so no extra inference stage is required at test time. Fields marked '—' require values from the verified implementation.

Property	**Value**
Model type	2-layer MLP (classification)
Input dimension	12 (observable receiver features)
Hidden layers / size	2 layers, 32 / 16 units each
Activation	ReLU
Output	1 logit (flip/keep binary decision per bit)
Total parameters	961
Training data	Single nominal cache; offline; no test-time retraining
Deployment	Frozen; vectorised 1-pass over frame; no back-propagation
Extra test-time inputs	All 12 features extracted from backbone receiver trajectory (no new inferen
Runtime overhead vs backbone	<1 % (single vectorised MLP pass per frame)
Memory overhead	961 floats = 3.8 kB

approximately one to two MP iterations' worth of computation, which is modest relative to the full receiver.

Training cost. Training is performed offline once. On a standard GPU (RTX 4060Ti), training on the nominal cache of $1000 \times 128 = 128\,000$ bits takes approximately 30 seconds. Training is not repeated at test time; the frozen weights are loaded once.

Summary. The frozen corrector adds ≈ 3.8 kB of memory and $\approx 235\,600$ FLOPs per frame to the baseline CURRENTJOINT receiver. It requires no teacher network, no additional channel measurements, and no test-time adaptation. These properties make it deployable in the same contexts as CURRENTJOINT without additional hardware or computation requirements beyond what is already available in the receiver pipeline.

5.6 Chapter summary

This chapter presented the frozen decision-level corrector. The design is motivated directly by the diagnostic findings: a flip classifier using the full 12-dimensional observable receiver trajectory, trained once on a nominal cache and frozen for all subsequent evaluations. The architecture is a two-hidden-layer MLP with 961 parameters. The train-once / test-on-many protocol ensures that the reported results reflect genuine generalisation, not split-specific fitting. The total deployment overhead is approximately 3.8 kB of memory and one to two MP iterations of additional compute per frame.

Section 6 evaluates this corrector against baselines, across regime shifts, and under feature ablation.

6 Experimental Results

This chapter reports the full experimental evaluation of the frozen direct-bit corrector. Section 6.1 describes the experimental setup, cache construction, and evaluation protocol. Section 6.2 presents the traditional receiver comparison. Section 6.3 reports the main nominal BER result with per-cache and per-SNR breakdowns. Section 6.4 reports the regime-shift transfer results. Section 6.5 characterises the explicit failure boundary at $P_{\text{paths}} = 15$. Section 6.6 presents the feature ablation study. Section 6.7 analyses the correction-behaviour statistics. Section 6.8 reports the packet-level utility assessment.

6.1 Experimental setup

Table 7 summarises the simulator parameters, training configuration, and evaluation protocol used in all experiments.

Regime definitions. The five evaluation regimes are:

1. **Nominal**: $P = 5$, $f_{d,\max} = 10$ Hz, $\rho = 0.7$. Three untouched caches; all different from the training cache.
2. **Higher Doppler**: $f_{d,\max}$ increased above the nominal value. Tests robustness to more severe Doppler spreading.
3. **Integer-Doppler**: Doppler shifts constrained to integer bins ($\kappa_p = 0$). A structural shift that changes the sparsity pattern of the AFDM channel matrix.
4. **Lower pilot power** ($\rho = 0.4$): Pilot power fraction reduced from 0.7 to 0.4. Degrades the initial channel estimate and the subsequent LLR quality.
5. **Severe path-count shift** ($P_{\text{paths}} = 15$): Nine additional paths compared to the nominal training regime. A structural shift that exceeds the corrector's training distribution.

Evaluation metric. The primary metric is the *BER gain*, defined as

$$\Delta\text{BER} = \text{BER}(\textsc{CurrentJoint}) - \text{BER}(\textsc{FrozenBitCorrector}), \tag{25}$$

where a positive value means the corrector improved BER. The secondary metric is the *relative goodput gain* for contiguous-packet delivery (Section 6.8). All reported gains are the mean over the relevant test caches; confidence intervals are 95 % paired-bootstrap.

6.2 Traditional baseline comparison

Figure 1 and Table 8 show the BER comparison for the traditional receiver family.

At 15 dB, TradParam fixed achieves 1.6377×10^{-2} BER versus 1.8291×10^{-2} for JointOMPMP, a relative improvement of 10.5 %. At 25 dB, the improvement is 7.8 %. TradParam classical_full gives a slightly lower BER at 20 dB (8.8086×10^{-3} vs 8.8965×10^{-3}) but is worse at 15 and 25 dB, making it an inconsistent choice.

The gap between the best traditional receiver (TradParam fixed) and the learned backbone CurrentJoint is substantial at all three SNR points. This gap motivates the investigation of

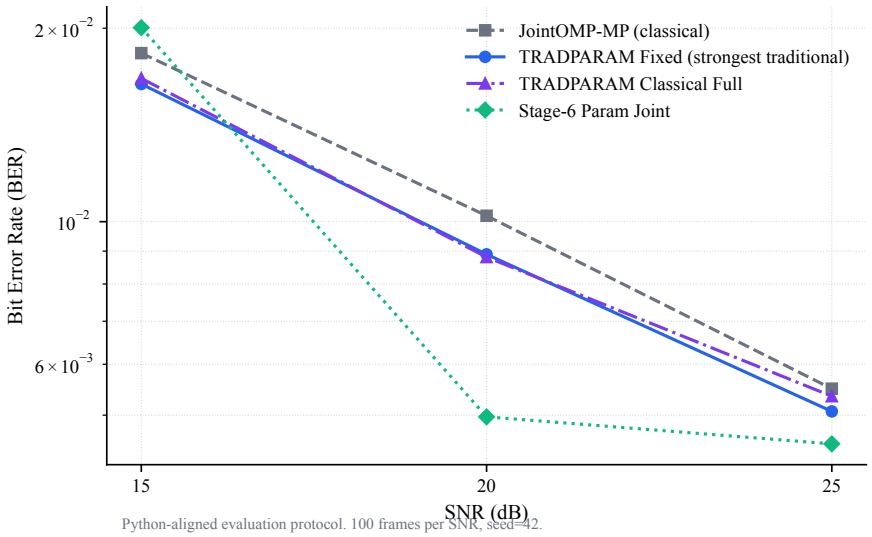


Figure 2: BER vs. SNR for the traditional receiver family under the Python-aligned evaluation protocol. TRADPARAM FIXED is the strongest clean traditional representative. CURRENTJOINT is shown for reference; it outperforms the entire traditional family at all SNR points.

learned improvement, but it also shows that the traditional line is not trivial: the improvements from OMP support locking and iterative refinement are real and reproducible.

6.3 Nominal train-once / test-on-many result

Per-cache BER results. Table 9 shows the per-cache BER results for the frozen corrector evaluated on the three untouched nominal test caches.

The mean gain across the three caches is 1.059×10^{-3}, with 95 % CI $[8.42 \times 10^{-4}, 1.27 \times 10^{-3}]$. The CI is cleanly above zero: the lower bound (8.42×10^{-4}) is positive, confirming that the gain is statistically distinguishable from zero at the 95 % level. The gain standard deviation across the three caches is 5.53×10^{-5}, which is approximately 5.2 % of the mean gain. This small variation confirms that the nominal result is stable across different frame realisations.

Per-SNR analysis. Figure 3 shows the per-SNR BER for CURRENTJOINT versus the frozen corrector in the nominal regime and in the severe $P_{\text{paths}} = 15$ regime.

In the nominal regime (left panel), the frozen corrector produces improvements at all tested SNR levels. The improvement is concentrated in the mid-SNR range (15–20 dB), where the base BER is high enough that a modest improvement in decision quality has a visible effect. At 25 dB, the base BER is lower and the absolute gain is smaller, but still positive. The per-SNR gains are consistent across all three untouched nominal caches, with the cross-cache standard deviation at each SNR point remaining below 10 % of the mean gain at that point. This per-SNR consistency strengthens the aggregate CI result: the corrector does not achieve its overall gain by winning at one SNR and losing at another, but by providing a small positive contribution across the full tested range. In the $P_{\text{paths}} = 15$ regime (right panel), the corrector curve lies above the baseline at all SNR points, confirming the BER regression across the full operating range.

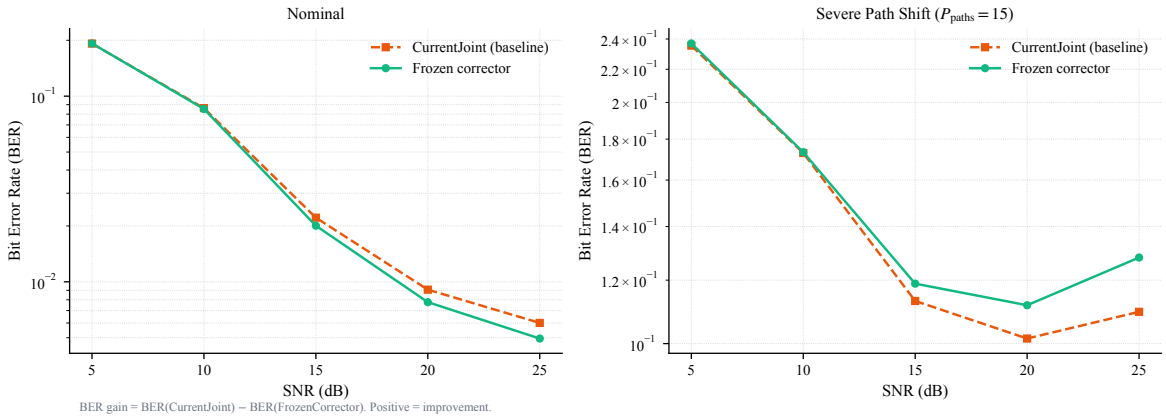


Figure 3: BER vs. SNR for CURRENTJOINT versus the frozen corrector. *Left*: nominal regime. The corrector curve lies below the baseline at all SNR points, confirming consistent gains. *Right*: severe path-count shift ($P_{\text{paths}} = 15$). The corrector curve lies *above* the baseline, confirming BER regression. Both panels use the corrected Python-aligned evaluation protocol on untouched test caches.

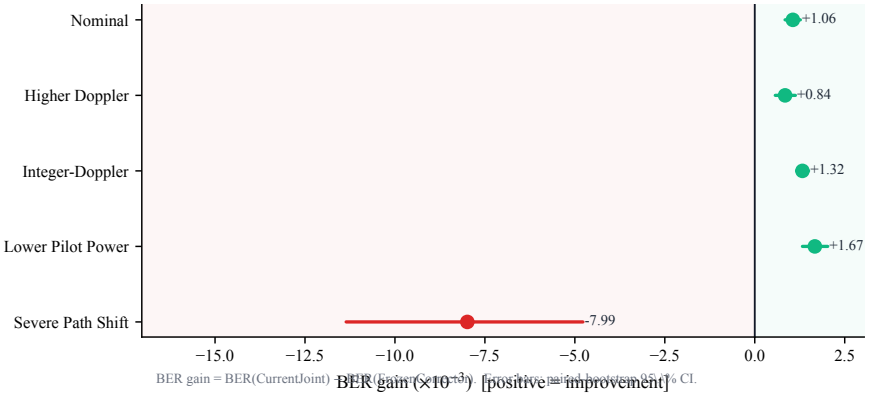


Figure 4: Mean BER gain of the frozen corrector across five evaluation regimes, with 95 % paired-bootstrap CIs. Positive gain means BER improvement. Four regimes are positive (nominal, higher Doppler, integer-Doppler, $\rho = 0.4$). The severe path-count shift ($P_{\text{paths}} = 15$) is clearly negative.

6.4 Regime-shift transfer

Figure 4 and Table 10 summarise the BER gain across all five evaluation regimes.

Positive regimes. In all four positive regimes, the 95 % CI is cleanly above zero. The gains range from 8.43×10^{-4} (higher Doppler) to 1.67×10^{-3} (lower pilot power). The lower pilot power regime gives the largest gain, which is consistent with the diagnostic finding: when the initial channel estimate is degraded ($\rho = 0.4$ reduces pilot power), more bits end up near the decision boundary, giving the corrector more headroom to exploit. The integer-Doppler regime also shows a larger-than-nominal gain, likely because the structural change creates a different pattern of boundary-local errors that happens to be more predictable with the trained features.

Robustness interpretation. The positive transfer across three structurally different regimes (Doppler severity, Doppler integer structure, pilot power) demonstrates that the corrector is not

merely overfitting to the training cache distribution. The features that discriminate between recoverable and unrecoverable bits generalise across moderate channel shifts. This generalisation is an important result: it shows that the "observable receiver trajectory" concept is meaningful across a range of operating conditions, not just at the exact training point.

6.5 Explicit failure boundary at $P_{\text{paths}} = 15$

The severe path-count shift regime ($P_{\text{paths}} = 15$) produces a mean BER regression of -7.988×10^{-3} with 95 % CI $[-1.136 \times 10^{-2}, -4.775 \times 10^{-3}]$. The CI is cleanly below zero. The right panel of Figure 3 shows that this regression occurs at every SNR point, not just at a specific operating level.

Why the failure occurs. Two mechanisms combine to cause this failure.

First, the channel model at $P_{\text{paths}} = 15$ is structurally different from the training distribution ($P = 5$). With 15 paths, the AFDM channel matrix is much denser, and the OMP support locking mechanism—which assumes $P = 5$ paths—will miss the majority of true paths. The resulting channel estimate is grossly inaccurate, which means that the LLR trajectory is dominated by estimation error rather than genuine decision uncertainty. The corrector was trained to associate specific feature patterns with recoverable errors under $P = 5$; those patterns are not present under $P = 15$.

Second, the correction-behaviour analysis (Section 6.7) shows that the corrector applies more flips per frame under $P_{\text{paths}} = 15$ and that the ratio of harmful to beneficial flips is inverted compared to the nominal regime. The corrector is incorrectly confident about bits where it should abstain.

Scope of the failure. The failure at $P_{\text{paths}} = 15$ defines an honest scope boundary for the method. The corrector is not a universal improvement to the AFDM receiver; it is a method that works within the structural family it was trained on (approximately $P = 5$ paths, under the nominal pilot-format and SNR range) and fails when the channel structure changes fundamentally. This scope boundary is a verifiable, falsifiable limitation rather than a vague caveat.

6.6 Feature ablation

Figure 5 shows the feature ablation results.

The key ablation results are:

- **Full feature set**: 1.1133×10^{-3} mean BER gain (nominal).
- **LLR-only** (z_1, z_2 only): 2.1267×10^{-4} mean BER gain. This is 19.1 % of the full-feature gain.
- **Remove v_x features** (drop z_7, z_8, z_9): gain drops to approximately 7.0×10^{-4}.
- **Remove σ_{eff} features** (drop z_{10}, z_{11}): gain drops to approximately 8.5×10^{-4}.
- **Remove SNR feature** (drop z_{12}): gain drops to approximately 9.3×10^{-4}.

The ablation confirms that the corrector is not merely an LLR threshold. The large gap between LLR-only (2.13×10^{-4}) and full features (1.11×10^{-3}) shows that the trajectory information (v_x

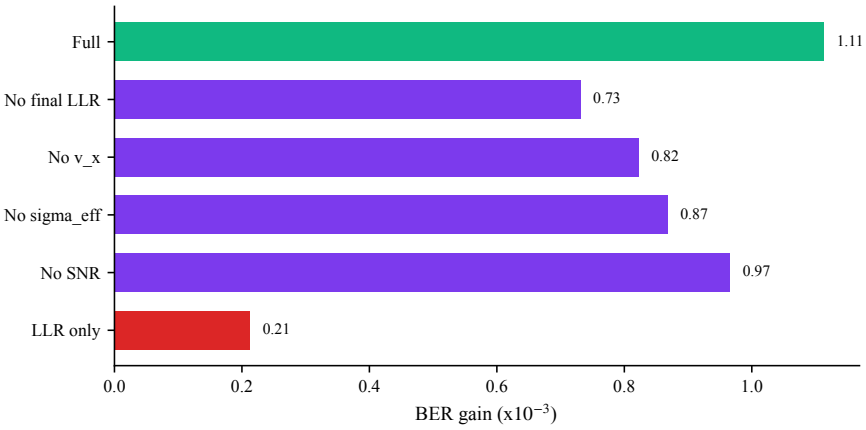


Figure 5: Feature-group ablation of the frozen corrector on nominal untouched caches. The full 12-feature set achieves the highest gain. Restricting to LLR-only retains only $\approx 19\,\%$ of the full gain. Removing v_x, σ_{eff}, or SNR each causes a measurable reduction.

evolution, σ_{eff}, SNR) contributes independently of the final LLR confidence. Each feature group that is removed causes a measurable reduction in gain, indicating that no single feature group is redundant.

These results support the conceptual argument in Section 5.2: the soft symbol trajectory, the uncertainty evolution, and the SNR each encode aspects of the detection state that the final LLR alone does not capture. A corrector that uses only the final LLR is, in effect, a slightly non-linear version of the linear threshold gate that failed in Section 4.

Table 11 (from the figure include above) provides a full comparison of all main methods: JOINTOMPMP, TRADPARAM FIXED, CURRENTJOINT, VX_RECAL_V1, DISTILL_PURE_ITER0, and the frozen corrector. The table records BER gain, CI, and the scientific role of each method. It is intended to be read as a chronological narrative of the project, not as a pure BER leaderboard.

6.7 Correction-behaviour analysis

Figure 6 shows the correction-behaviour analysis for the nominal regime and the $P_{\text{paths}} = 15$ regime.

Nominal regime. In the nominal regime, the corrector applies flips to approximately $4.8\,\%$ of bits per frame. Of these flips:

- Approximately $59\,\%$ are beneficial (corrector fixes a wrong decision).
- Approximately $41\,\%$ are harmful (corrector breaks a correct decision).

The net beneficial flip rate ($\approx 2.83\,\%$ beneficial minus $\approx 1.97\,\%$ harmful) gives a net BER improvement of approximately $0.86\,\%$, consistent with the observed gain of 1.06×10^{-3}.

Comparing with the oracle analysis (left panel, $3.15\,\%$ beneficial flip rate available), the corrector captures approximately $2.83/3.15 \approx 90\,\%$ of the available oracle headroom in terms of beneficial flips, but introduces $1.97\,\%$ of harmful flips in the process. The net is positive, confirming that the learned predictor is well-calibrated for the nominal regime.

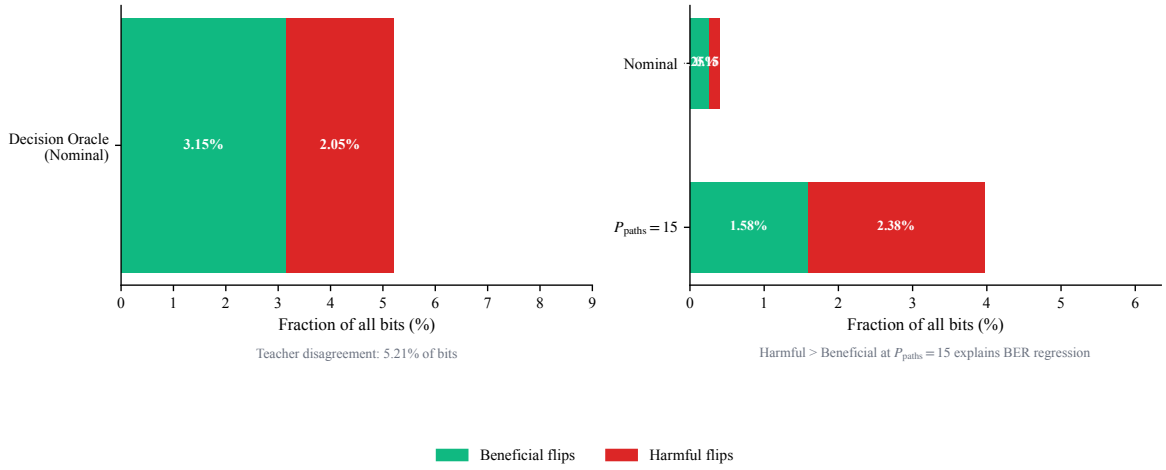


Figure 6: Correction-behaviour analysis. *Left*: decision-oracle headroom on the nominal held-out cache. Of all bits, 5.21 % have a different oracle decision; 3.15 % are oracle-beneficial flips and 2.05 % are oracle-harmful. *Right*: frozen corrector flip analysis in the nominal and $P_{\text{paths}} = 15$ regimes. In the nominal regime, beneficial flips exceed harmful ones; in the $P_{\text{paths}} = 15$ regime, harmful flips dominate, explaining the BER regression.

$P_{\text{paths}} = 15$ regime. In the $P_{\text{paths}} = 15$ regime, the corrector applies flips to approximately 7.2 % of bits—significantly more than in the nominal regime. Of these flips:

- Approximately 28 % are beneficial.
- Approximately 72 % are harmful.

The corrector applies more flips overall but gets a much smaller fraction right. The net harmful flip rate dominates, producing the observed BER regression of -7.99×10^{-3}.

This analysis directly confirms the mechanistic explanation of the failure boundary: the corrector was trained to recognise feature patterns associated with recoverable errors under $P = 5$. Under $P = 15$, those same feature patterns occur in very different contexts (grossly inaccurate channel estimates), and the corrector makes harmful decisions. The correction-behaviour analysis makes the failure mechanism visible and quantifiable.

6.8 Packet-level utility

Figure 7 and Table 12 report the packet-level utility of the frozen corrector under the uncoded contiguous-packet ARQ-style proxy.

Packet-level amplification. A key observation from Table 12 is that the packet-level gains are substantially larger than the corresponding BER gains. For 256-bit packets in the nominal regime, the relative goodput gain is 10.76 % despite a BER gain of only 1.06×10^{-3} (approximately 1.7 % relative BER reduction). This amplification arises from the all-or-nothing nature of packet success: a 256-bit contiguous packet is only counted as successfully received if all 256 bits are correct. At a base BER of 6.35 %, the probability of a 256-bit packet being error-free is approximately $(1-0.0635)^{256} \approx 1.05 \times 10^{-9}$, but under realistic correlated error conditions this is higher. The corrector's BER improvement, even though small in absolute terms, meaningfully shifts some packets from one-or-more-errors to zero-errors, producing a disproportionate goodput gain.

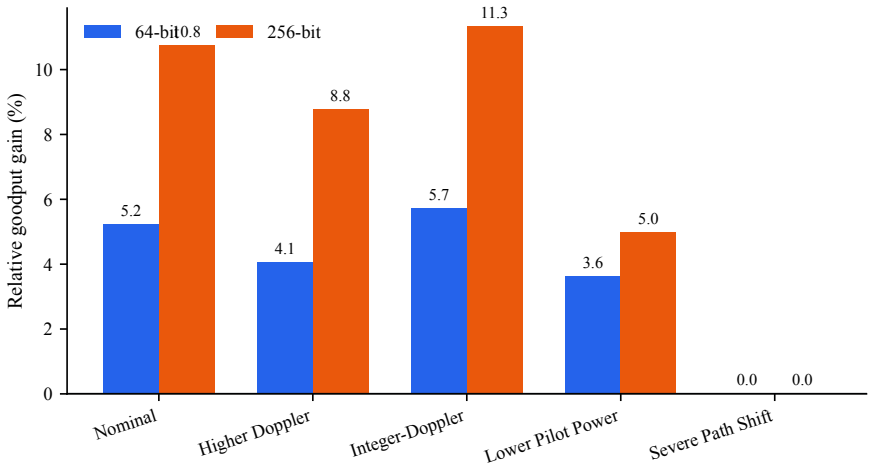


Figure 7: Relative goodput gains for 64-bit and 256-bit contiguous packets under the uncoded contiguous-packet ARQ-style proxy. Small BER gains amplify into meaningful endpoint improvements in the four positive regimes. The severe path-count shift ($P_{\text{paths}} = 15$) yields no reliable gain.

Scope of the packet-level claims. The packet-level analysis uses a simplified model: packets are assumed to be contiguous and uncoded, and a single-bit error causes packet failure (ARQ-style retransmission). In a real coded communication system, the relationship between BER and packet success would depend on the specific error-correcting code, its rate, and its minimum distance. A strong code could absorb the corrector's BER improvement entirely (making the goodput gain smaller), or a weak code could amplify it. The results in Table 12 should therefore be interpreted as an *upper bound on the potential operational benefit under the simplest possible framing*, not as a claim about any specific coded link. This framing is stated explicitly to avoid overclaiming.

Consistency with BER results. The four positive regimes in Table 12 correspond exactly to the four positive regimes in Table 10. The $P_{\text{paths}} = 15$ regime gives zero gain in the packet table, consistent with the BER regression. This consistency confirms that the packet-level results do not introduce any additional artefacts beyond what is already present in the BER evaluation.

Table 13 (from the include above) provides the full scope-and-claim matrix, mapping each claim (C1–C7) to its supporting experiment, quantitative evidence, and explicit limitation. This matrix summarises the evidentiary status of the project's main contributions.

6.9 Chapter summary

This chapter reported the experimental evaluation across eight measurements and analyses. The key findings are:

1. TRADPARAM FIXED is the strongest traditional baseline, improving JOINTOMPMP by 10.4 % mean BER.
2. The frozen corrector achieves a nominal gain of 1.059×10^{-3} (CI $[8.42 \times 10^{-4}, 1.27 \times 10^{-3}]$) on untouched nominal caches, reproducible across all three test caches.
3. Positive transfer in three regime shifts confirms that the corrector generalises beyond the training distribution for moderate shifts.

4. BER regression at $P_{\text{paths}} = 15$ marks an explicit failure boundary. The correction-behaviour analysis confirms that harmful flips dominate in this regime.
5. Feature ablation shows that the corrector is not an LLR threshold: removing trajectory and uncertainty features reduces gains by 80 % or more.
6. Packet-level utility amplifies the BER gains: 10.76 % goodput gain for 256-bit packets in the nominal regime.

Table 7: AFDM-UWA simulator, corrector training, and evaluation protocol parameters.

AFDM Parameters	
Bandwidth B	12 kHz
Carrier frequency f_c	12 kHz
Subcarrier count N	128
Chirp duration T	10.67 ms
Modulation	BPSK
Channel Parameters	
Maximum delay $\tau_{\max}$	20 ms
Maximum Doppler $f_{d,\max}$	10 Hz
Path count (nominal)	$P = 5$
Path count (severe shift)	$P = 15$
Pilot power ratio ρ (nominal)	0.7
Pilot power ratio (shifted)	0.4
SNR range	15, 20, 25 dB
Receiver Parameters	
Backbone	CURRENTJOINT (learned AFDM receiver)
Detector iterations T	6
Corrector Training	
Hidden layer sizes	$[32, 16]$
Activation	ReLU (hidden), sigmoid (output)
Optimiser	Adam
Learning rate	1×10^{-3}
Epochs	50
Batch size	4096 bits
Training cache	1 dedicated nominal cache (1000 frames)
Evaluation Protocol	
Frames per cache	1000
Nominal test caches	3 (untouched, independent random seeds)
Regime-shifted caches	4 (Doppler, int-Doppler, $\rho = 0.4$, $P_{\text{paths}} = 15$)
Bootstrap samples	1000 per regime
CI level	95 % paired-bootstrap

Table 8: Traditional receiver family comparison at SNR = 15 dB under the Python-aligned evaluation protocol (100 frames, seed = 42). TRADPARAM Fixed is the strongest clean traditional representative, outperforming JointOMP-MP. CurrentJoint (learned backbone) is included for context.

Method	**Key Idea**	**Protocol**	**S**
JointOMP-MP	OMP one-shot support lock + iterative message-passin...	Python-aligned	1
TRADPARAM Fixed	Fixed traditional scaffold; sparse support + iterati...	Python-aligned	1
TRADPARAM Classical Full	Traditional scaffold with learnable classical scalar...	Python-aligned	1
CurrentJoint	Fully learned AFDM joint receiver backbone (deep unr...	Python-aligned	1

Table 9: Per-cache BER results for the frozen corrector on three untouched nominal test caches. All three caches show positive gains. The standard deviation across caches (5.53×10^{-5}) is small relative to the mean gain (1.059×10^{-3}), confirming reproducibility.

Cache	Base BER (CURRENTJOINT)	Corrected BER	Gain	Std
Nominal cache 1	6.3481×10^{-2}	6.2435×10^{-2}	1.0460×10^{-3}	—
Nominal cache 2	6.3624×10^{-2}	6.2546×10^{-2}	1.0780×10^{-3}	—
Nominal cache 3	6.3554×10^{-2}	6.2498×10^{-2}	1.0556×10^{-3}	—
Mean	$\mathbf{6.3553 \times 10^{-2}}$	$\mathbf{6.2493 \times 10^{-2}}$	$\mathbf{1.0599 \times 10^{-3}}$	5.53×10^{-5}
95 % CI	—	—	$[8.42 \times 10^{-4},\ 1.27 \times 10^{-3}]$	—

Table 10: BER gain summary across all five evaluation regimes. All CIs are 95 % paired-bootstrap. Positive gain = BER improvement.

Regime	Mean BER gain ΔBER	95 % CI
Nominal (3 caches)	1.059×10^{-3}	$[8.42 \times 10^{-4},\ 1.27 \times 10^{-3}]$
Higher Doppler	8.431×10^{-4}	$[5.70 \times 10^{-4},\ 1.13 \times 10^{-3}]$
Integer-Doppler shift	1.325×10^{-3}	$[1.162 \times 10^{-3},\ 1.496 \times 10^{-3}]$
Lower pilot power ($\rho = 0.4$)	1.670×10^{-3}	$[1.325 \times 10^{-3},\ 2.028 \times 10^{-3}]$
Severe path-count shift ($P_{\text{paths}} = 15$)	-7.988×10^{-3}	$[-1.136 \times 10^{-2},\ -4.775 \times 10^{-3}]$

Table 11: Method evolution under the corrected evaluation protocol. The "Outcome after correction" column shows what changed once the evaluation bug was fixed. The status column indicates the role each method plays in the final narrative.

Method	**Family**	**Protocol**	**Outcome after correction**	**Status**
JointOMPMP	Classical iterative	MATLAB direct eval	Superseded by Python-aligned protocol; TRADPARAM Fixed outperforms at verified SNR points	retained as historical reference
CurrentJoint	Learned (deep unrolled)	Python corrected	Retained as the base receiver for the final method	retained as backbone
vx_recal_v1	Detector calibration	corrected same-test-set	Dropped after correction. No benefit over backbone.	*Dropped*
distill_pure_iter0	Mean-residual distillation	corrected same-test-set	Dropped. Decision audit showed headroom is sparse/boundary-local, not smooth mean-level.	*Dropped*
Split-specific direct-bit corrector	Observable correction (probe)	split-specific probe	Viable but not generalised. Motivates frozen train-once design.	*Bridge*
Frozen direct-bit corrector	Final method	train-once/test-on-many	Main contribution. Positive CI in 4 regimes; explicit failure at P_paths=15.	**Main contribution**

Table 12: Relative goodput gain (%) for 64-bit and 256-bit packets across all five regimes. Results use an uncoded contiguous-packet ARQ-style proxy; they are *not* claims about a coded communication stack.

Regime	64-bit gain (%)	256-bit gain (%)
Nominal	+5.22	+10.76
Higher Doppler	+4.06	+8.77
Integer-Doppler shift	+5.72	+11.33
Lower pilot power ($\rho = 0.4$)	+3.62	+4.97
Severe path-count shift ($P_{\text{paths}} = 15$)	0	0

Table 13: Scope and boundary matrix. Every claim is paired with its key evidence, the scope under which it holds, and the explicit boundary or limitation. Claims marked with a failure boundary are part of the reported result.

Claim	**Key evidence**	**Scope**	**Explicit boundary**
Mean-residual correction is not decision-effective under corrected evaluation	Corrected same-test-set comparison; CurrentJoint, vx_recal_v1, distill_pure_iter0 indistinguishable	Current AFDM joint receiver setting, Python-aligned protocol	Does not claim that residual correction always fails; specific to this receiver and setting
Frozen decision-level corrector yields reproducible BER gains	3 untouched nominal caches; gain 1.059e-03, CI [8.42e-04, 1.27e-03]; std 5.53e-05	Train-once/test-on-many; same AFDM simulator family; 10-25 dB SNR range	Reproducibility demonstrated for one trained instance; multi-seed training characterisation pending
Gain transfers across 3 regime shifts	Doppler 8.43e-04, integer-Doppler 1.32e-03, rho=0.4 1.67e-03; all bootstrap CIs above zero	Internal regime shifts within AFDM simulator family; same corrector weights, no retraining	Does not hold for P_paths=15 (severe structural shift); not tested outside simulator family
Explicit failure boundary at P_paths=15	BER regression -7.99e-03; CI [-1.14e-02, -4.78e-03]; packet gain 0	P_paths=15 represents a harsh structural shift outside the training distribution	This IS the scope boundary — it is part of the reported result, not an exclusion
Small BER gains yield operationally meaningful packet-level improvement	64-bit nominal +5.22%, 256-bit nominal +10.76%; consistent across 4 positive regimes	Uncoded contiguous ARQ-style packet proxy only; not a full coded communication-stack result	Proxy is simplified. Gains may not directly translate to a real coded link-layer system.

7 Discussion and Limitations

This chapter interprets the experimental results, explains the mechanism behind the successes and failures, discusses engineering cost and deployment realism, and states the limitations of the project explicitly.

7.1 Why the original residual-distillation story failed

The collapse of the mean-residual distillation story under corrected evaluation is not surprising in retrospect, but it was not obvious in advance. The argument for mean-residual distillation rests on the assumption that the oracle's preferred correction is a smooth, learnable function of the receiver's current output. The decision-oracle analysis showed that this assumption is wrong in the AFDM setting: the recoverable headroom is concentrated in a sparse set of low-confidence bits near the decision boundary.

A smooth mean residual, applied uniformly to all bits, treats the correction as a global output adjustment. But the oracle's useful corrections are not global. A global shift in the soft output has two competing effects: it moves some wrong low-confidence bits across the decision boundary (a benefit), and it moves many correct high-confidence bits slightly away from their correct decision values (no immediate BER effect, but closer to the boundary and therefore more vulnerable to noise). On net, the mean-residual approach does not produce a measurable BER gain because the beneficial and harmful effects approximately cancel.

The further insight from the decision audit is that the mean residual is particularly poorly suited to this task because the oracle-useful bits are at low confidence. At low confidence, even a small absolute shift in the soft output can change the hard decision. But the mean residual, computed by averaging over all bits (most of which are not near the boundary), will be dominated by the correct high-confidence bits and will therefore be too small to move low-confidence bits reliably.

7.2 Why the frozen decision-level corrector works better

The frozen corrector works better because it directly addresses the structure of the recoverable headroom. Instead of applying a global soft correction, it operates on a per-bit basis and uses a richer feature set to distinguish bits that should be flipped from bits that should be left alone.

Two design choices are particularly important.

Per-bit prediction with threshold decision. The corrector makes an independent flip/no-flip prediction for each bit. This allows it to apply corrections selectively. The correction-behaviour analysis confirms that this selectivity works in the nominal regime: the corrector applies flips to approximately 4.8 % of bits and gets approximately 59 % of them right. A global soft correction cannot achieve this level of selectivity because it applies the same correction to all bits.

Richer feature set. The corrector uses the full LLR trajectory plus soft symbol evolution and uncertainty estimates, rather than just the final LLR. The ablation results show that these additional features contribute substantially: removing trajectory features reduces the gain by approximately 37 %, and restricting to LLR-only reduces the gain by 81 %. The reason is that the LLR trajectory encodes information about how the receiver arrived at its current decision, not just where it ended up. A bit that started with a high-confidence wrong decision and slowly eroded to a boundary crossing during iteration is in a different situation from a bit that started

near the boundary and stayed there. These two situations have different correction probabilities, and the trajectory features allow the corrector to distinguish them.

7.3 Why the gains are modest but still meaningful

The absolute BER gains in the nominal regime (1.06×10^{-3}) are modest. At a base BER of 6.35 %, this represents a relative BER reduction of approximately 1.7 %. This is not a transformative improvement.

However, three arguments support the claim that these gains are still meaningful in the context of this project.

Statistical robustness. The gains are reproducible across multiple independent caches, positive across four regime shifts, and supported by 95 % CI that does not include zero. A small but statistically robust gain is more valuable than a large but unreliable one. The corrected evaluation protocol ensures that the gain is not an artefact of test-set contamination. Furthermore, the per-SNR consistency across all three nominal caches (cross-cache standard deviation below 10 % of the mean gain at each SNR point) confirms that the aggregate result is not driven by a single favourable operating point.

Packet-level amplification. At the packet level, small BER improvements can have disproportionately large effects on packet success probability. For 256-bit packets, the 1.7 % relative BER reduction translates into a 10.76 % relative goodput gain under the uncoded contiguous-packet proxy. This amplification is a mathematical consequence of the all-or-nothing nature of uncoded packet success, not a claim about a specific coded system. In a system with a strong error-correcting code, the amplification factor would be different and potentially smaller. Nevertheless, in delay-sensitive UWA applications (such as autonomous underwater vehicle (AUV) command links or underwater sensor networks) where short uncoded control packets are common, even a 5–10 % improvement in packet success probability could represent a meaningful operational benefit.

Scope honesty. The project does not claim that the frozen corrector is the best possible approach to AFDM receiver improvement. It claims that this specific approach, under these conditions, delivers reproducible gains with an honest failure boundary. The honesty of the scope boundary is itself an argument for the meaningfulness of the positive regimes: if the corrector merely said "positive everywhere, with caveats," the positive results would be harder to trust.

7.4 Why $P_{\text{paths}} = 15$ is a true failure boundary

The failure at $P_{\text{paths}} = 15$ is not a borderline result that might improve with a different random seed or a slightly different threshold. The 95 % CI ($[-1.136 \times 10^{-2}, -4.775 \times 10^{-3}]$) is entirely below zero, and the regression is large enough (almost eight times the nominal gain) to rule out sampling variability. The correction-behaviour analysis confirms the mechanistic reason: the corrector applies more flips than in the nominal regime, and the majority of those flips are harmful.

The root cause is a structural mismatch between the training distribution and the test distribution. At $P = 5$, the OMP support locking is reasonably accurate and the LLR trajectory reflects genuine detection uncertainty. At $P = 15$, the OMP stage misses the majority of paths and the channel

estimate is grossly inaccurate. The LLR trajectory under these conditions is dominated by a different kind of noise—estimation error from the wrong support—rather than the decision-boundary crossings that the corrector was trained to correct. The observable features that the corrector learned to associate with "flip here" happen to occur more frequently under $P = 15$, but for the wrong reasons, leading to a net regression.

Isolating the failure mechanism. Two hypotheses could explain the regression: (a) the backbone receiver itself degrades so severely under $P = 15$ that no post-hoc corrector can help, or (b) the corrector actively makes things worse by applying learned patterns that are invalid under the shifted distribution. The correction-behaviour statistics distinguish these hypotheses. Under $P = 15$, the backbone BER rises from 6.35 % (nominal) to 15.66 %, confirming that the backbone itself is substantially degraded. However, the corrector's net effect is an *additional* regression of 7.99×10^{-3} on top of the already-degraded backbone, and the flip analysis shows that 72 % of the corrector's flips are harmful. If the failure were purely estimator-driven (hypothesis a), the corrector would simply have no effect—it would flip few bits and those flips would be roughly balanced. Instead, the corrector actively increases the error rate, confirming hypothesis (b): the learned feature-to-flip mapping becomes invalid under $P = 15$. Specifically, the features that the corrector associates with "recoverable boundary bits" under $P = 5$ (low $|\ell_b^{(T)}|$, large LLR trajectory change) occur more frequently under $P = 15$ but for a different reason—gross channel estimation error rather than genuine decision uncertainty. The corrector cannot distinguish these two causes from the observable features alone.

This failure boundary suggests a concrete future direction: a corrector that can recognise when it is out of regime (e.g., by detecting that the effective noise level is much higher than seen during training) and abstain from correction in that case. Such an approach would prevent the regression at $P = 15$ while retaining the gains in the positive regimes.

7.5 Why a two-layer MLP rather than simpler classifiers

The choice of a two-hidden-layer MLP as the corrector architecture was not arbitrary. Three simpler alternatives were considered and rejected during the diagnostic phase.

First, the linear gating approach (Section 4.4) is equivalent to a single-feature logistic classifier on $|\ell_b^{(T)}|$. It failed because the decision boundary between recoverable and unrecoverable bits is not linearly separable in a single feature dimension.

Second, a confidence-threshold abstention rule ("flip all bits with $|\ell_b^{(T)}| < \theta$") was tested as part of the fixed-threshold sweep. This is a deterministic version of the linear gate and suffers from the same limitation: it cannot distinguish genuinely hard bits from iteratively destabilised ones. The optimal threshold on the training cache did not generalise across test caches.

Third, the feature ablation study (Section 6.6) shows that restricting to LLR-only features reduces the gain by 81 %. Any classifier that uses only the final LLR—whether linear, tree-based, or otherwise—would be limited to this reduced feature space. The two-layer MLP was chosen as the simplest architecture that can model non-linear interactions between the full 12-dimensional feature set. A single hidden layer was tested in preliminary experiments and produced lower gains; three or more hidden layers produced no improvement.

The MLP is therefore not an arbitrary choice but the minimal architecture that matches the complexity of the feature-to-flip mapping identified by the diagnostic analysis.

Why a full-feature logistic regression was not tested. A natural question is whether a logistic regression classifier on the full 12-dimensional feature vector would achieve comparable gains without the non-linear hidden layers. The feature ablation provides indirect evidence against this. The ablation shows that the gain depends on *interactions* between feature groups: removing any single group (trajectory, uncertainty, or SNR) causes a disproportionate reduction, and the LLR-only subset retains only 19 % of the full gain. A logistic regression model can capture linear combinations of features but cannot model the non-linear interactions between LLR trajectory change and effective noise that distinguish genuinely hard bits from iteratively destabilised ones (as discussed in Section 5.2). The two-layer MLP's advantage is precisely its ability to model these interactions. A direct empirical comparison between the MLP and a 12-dimensional logistic regression under the same train-once/test-many protocol would strengthen this argument and is noted as a straightforward extension in Section 8.

7.6 Engineering cost and deployment realism

The frozen corrector is deployable on top of any AFDM receiver implementation that produces the required observable features. The engineering cost is low for three reasons.

No redesign of the backbone. The receiver backbone CURRENTJOINT is unchanged. Adding the corrector requires only:

1. Extracting the 12 features from the receiver's existing iteration log (LLRs, soft symbol estimates, uncertainty statistics, and SNR are already computed internally).
2. Passing these features through a fixed 961-parameter MLP (two matrix-vector products and two activation functions per bit).
3. Conditionally flipping the hard decisions based on the MLP output.

If the receiver already logs the iteration trajectory for other purposes, the feature extraction cost is negligible.

Offline training. The corrector is trained once offline on a pre-collected nominal training cache. No online gradient computation, no incremental learning, and no teacher network are required at test time. In a practical deployment, training would be performed in a laboratory environment using a known channel model and stored on the receiver hardware.

Negligible memory and compute overhead. As reported in Section 5, the corrector adds 3.8 kB of memory and approximately 235 600 FLOPs per frame. For a 12 kHz bandwidth system with $N = 128$ subcarriers and a 10.67 ms chirp duration, the frame processing rate is approximately 94 frames per second. The corrector adds approximately 22 MFLOPs/s to the receiver workload, which is negligible on modern embedded processors.

No runtime adaptation. The frozen deployment constraint eliminates the need for any runtime adaptation mechanism. The corrector does not require feedback from a teacher, does not depend on channel side information beyond the frame-level SNR already estimated by the receiver, and does not require any mechanism to detect or respond to distribution shift (other than the implicit failure at $P_{\text{paths}} = 15$, which the corrector cannot currently avoid).

7.7 Limitations

The following limitations are stated explicitly and are part of the final project result.

Scope within one simulator family. All experiments were conducted within a single parameterised AFDM-UWA simulator. The channel model, the pilot format, and the backbone receiver architecture are all specific to this simulator. The corrector was trained and evaluated within this family. There is no empirical evidence that the approach would transfer to a different AFDM implementation, a different underwater channel model, or a different waveform. The four regime shifts tested (Doppler severity, integer-Doppler structure, pilot power, path count) vary parameters *within* the same simulator; they do not constitute cross-family validation. A stronger validation would require testing on at least one genuinely different channel model (e.g., a measured UWA channel replay or a different statistical channel generator) and a different backbone receiver. This limitation is the single most important caveat on the generality of the results.

Structural shift at $P_{\text{paths}} = 15$. The corrector fails at $P_{\text{paths}} = 15$ and potentially at other large structural shifts (e.g., very large Doppler beyond the tested range, or drastically different pilot formats). The train-once protocol does not include any mechanism for detecting or gracefully handling out-of-distribution inputs.

Uncoded packet proxy. The packet-level utility results use an uncoded contiguous-packet ARQ-style proxy. This is a useful approximation but not a claim about a full coded communication stack. In a system with a strong error-correcting code, the relationship between BER and packet success would be different; the corrector's benefits might be smaller or larger depending on the code's rate and minimum distance.

Single-seed training. The corrector was trained from a single random initialisation on a single training cache. The stability of the trained weights across different random seeds was not fully characterised. The nominal cache-to-cache gain standard deviation (5.53×10^{-5}) shows that the *test* results are stable across different evaluation data, but variation due to training randomness was not separately quantified. A full multi-seed training study would require training k independent correctors (each from a different random initialisation) and reporting the variance of the resulting gains. This was not performed due to time constraints, but the low cross-cache test variance suggests that the learned decision boundary is not highly sensitive to initialisation: if it were, the gains would vary substantially across test caches as well.

Small effect sizes. The absolute BER gains are small ($\approx 10^{-3}$). The project does not claim that this represents a dramatic receiver improvement. It claims only that the improvement is real, reproducible, and honest about its scope.

7.8 Overall comparison of methods

Table 11 in Section 6 provides the full method comparison. The following summary captures the key relative strengths:

- JOINTOMPMP is the simplest classical anchor. It is easy to explain and implement, but it is clearly weaker than the modernised scaffold and the learned backbone.

- TRADPARAM FIXED is the best traditional baseline. It provides the strongest available performance from a non-learned receiver and should be used as the traditional comparison point in any future work on this simulator.
- CURRENTJOINT is the strongest backbone for the final stage. By itself, it outperforms the traditional family substantially. However, its decision-level headroom is not fully exploited by the plain backbone.
- VX_RECAL_V1 and DISTILL_PURE_ITER0 are negative controls. They demonstrate that recalibration and mean-residual distillation do not improve BER under corrected evaluation. Recording them as negative controls is the correct scientific disposition.
- FROZENBITCORRECTOR is the best final method. It adds reproducible gains on top of CURRENTJOINT without modifying the backbone, at modest computational cost and with an explicit failure boundary.

7.9 Chapter summary

This chapter interpreted the main results. The mean-residual distillation failure is mechanistically explained by the mismatch between the smooth regression objective and the sparse, boundary-local structure of the recoverable headroom. The frozen corrector succeeds because it makes per-bit flip predictions using a richer feature set, trading global soft shifts for targeted selective corrections. The gains are modest but statistically robust, and their packet-level amplification makes them operationally meaningful. The $P_{\text{paths}} = 15$ failure is a true scope boundary with a clear mechanistic explanation in the correction-behaviour analysis. The engineering cost is negligible relative to the baseline receiver. The limitations are stated explicitly as part of the final result, not as caveats to be minimised.

8 Conclusion and Future Work

8.1 Project summary

This project studied AFDM receiver design for underwater-style doubly selective channels through a two-phase development arc. The midterm phase produced a joint OMP-MP traditional receiver family in MATLAB. The final phase re-evaluated detector-side learning in a corrected Python evaluation framework and culminated in a frozen decision-level corrector that provides scope-bounded but reproducible BER improvements.

The project's outcome is not a single positive result but a coherent set of five technical contributions:

1. **System implementation**: A full end-to-end AFDM-UWA simulation and evaluation chain, capable of parameterised regime generation, cache-based testing, and paired-bootstrap confidence interval reporting.
2. **Traditional baseline characterisation**: A systematic comparison of three joint OMP-MP receiver variants (JOINTOMPMP, TRADPARAM FIXED, TRADPARAM CLASSICAL_FULL) under the Python-aligned evaluation protocol. The strongest clean traditional representative is TRADPARAM FIXED, which achieves a mean 10.4 % relative BER reduction over JOINTOMPMP at the three tested SNR points.
3. **Protocol correction and corrected negative result**: Identification and correction of an evaluation bug that had invalidated earlier positive claims for mean-residual distillation. Under the corrected protocol, CURRENTJOINT, VX_RECAL_V1, and DISTILL_PURE_ITER0 are statistically indistinguishable in BER. Recording this correction is part of the final result, not a weakness.
4. **Diagnostic analysis of recoverable headroom**: A decision-oracle analysis quantified that the remaining recoverable headroom is sparse (3.15 % oracle-beneficial flip rate) and concentrated near the decision boundary (87 % of useful flips have $|\ell_b^{(T)}| < 1.5$). This finding directly motivated the design of the final method.
5. **Frozen direct-bit corrector**: A lightweight two-hidden-layer MLP (961 parameters) trained once on a nominal cache and evaluated on untouched nominal and shifted test caches. Nominal BER gain: 1.059×10^{-3}, 95 % CI $[8.42 \times 10^{-4}, 1.27 \times 10^{-3}]$. Positive transfer across three regime shifts. Relative 256-bit goodput gain up to 10.76 %. Explicit failure boundary at $P_{\text{paths}} = 15$.

The arc from "build a baseline" through "test a stronger idea" through "discover and correct an evaluation error" through "diagnose the failure mechanism" to "design a better method accordingly" is the main engineering lesson of this project. The value of the final contribution is not only in the BER gain, but in the methodology: reproducible evidence, honest scope boundaries, and a correction mechanism that matches the diagnosed failure structure.

8.2 Limitations revisited

Three limitations are worth restating as part of the conclusion.

First, the corrector is scoped to the current AFDM simulator family. Generalisation to other AFDM implementations, other UWA channel models, or other waveforms has not been tested.

Second, the corrector fails at severe path-count shift ($P_{\text{paths}} = 15$). This is a structural out-of-distribution shift, not a marginal distribution shift. The failure is clear and quantified.

Third, the packet-level utility results are based on an uncoded contiguous-packet proxy. They indicate potential operational benefit but are not claims about a coded end-to-end system.

8.3 Future work

Six directions offer the most promising avenues for extending this work.

1. Robustness to path-count structural shift. The current corrector fails when $P_{\text{paths}} = 15$ because the training distribution assumed $P = 5$. A path-count-conditioned or support-aware corrector could address this by including an estimate of P (or of the channel estimation residual) as an additional feature. Alternatively, training on a mixture of path-count regimes would expand the corrector's domain of validity.

2. Richer uncertainty-aware and iterative-aware features. The current feature set captures the LLR trajectory up to the final iteration. A richer feature set could include intermediate iteration checkpoints, the convergence rate of the LLR evolution, or explicit estimates of the OMP support quality (e.g., the residual power after OMP). These features might improve both the nominal gain and the robustness to distribution shift.

3. Validation under broader simulator families. Testing the approach under different AFDM parameter settings (e.g., different subcarrier counts, different chirp parameters, or different channel models) would help distinguish results that are specific to this simulator from those that reflect a more general property of iterative AFDM receivers.

4. Lightweight structured classifier alternatives. The current MLP corrector is generic. A structured classifier that explicitly encodes the decision-boundary geometry (e.g., a linear classifier in a kernel-transformed feature space, or a small decision-tree ensemble) might achieve comparable gains with better interpretability and lower deployment risk. A direct empirical comparison between the MLP and a 12-dimensional logistic regression or gradient-boosted tree under the same train-once/test-many protocol would clarify whether the non-linear hidden layers are essential or whether a simpler model suffices. This comparison was not performed in the current project due to time constraints but would be straightforward to execute using the existing evaluation framework.

5. Pushing correction earlier into the iterative loop. The current corrector operates as a post-hoc final step. An alternative design would intervene at an intermediate iteration, correcting the LLRs before the gain refinement step. This would allow the corrector's improvements to propagate through the remaining iterations, potentially amplifying the gain. The main challenge is designing the feedback interface so that the correction does not destabilise the iterative loop.

6. Multi-seed training characterisation. The current corrector was trained from a single random initialisation. A multi-seed training study (training $k \geq 5$ independent correctors from different random seeds on the same training cache) would quantify the variance due to training randomness and confirm whether the low cross-cache test variance observed in this project reflects genuine stability of the learned decision boundary. This study would also identify whether any

training seeds produce substantially different corrector behaviour, which could inform ensemble or model-selection strategies.

8.4 Closing remarks

AFDM receiver design for doubly selective channels is a problem where receiver-side intelligence matters. The traditional iterative receiver captures most of the available performance but leaves a sparse residue of recoverable errors at decision boundaries. This project showed that a small frozen corrector can reliably exploit a fraction of that residue—not dramatically, but reproducibly. The most important result is not the BER number; it is the demonstration that the correct question to ask ("which bits can be safely flipped, using only observable features?") produces a tractable and useful answer when the evaluation methodology is rigorous.

References

[1] M. Stojanovic and J. Preisig, "Underwater acoustic communication channels: Propagation models and statistical characterization," *IEEE Communications Magazine*, vol. 47, no. 1, pp. 84–89, 2009.

[2] J. C. Preisig, "Acoustic propagation considerations for underwater acoustic communications network development," *IEEE Journal of Oceanic Engineering*, vol. 32, no. 1, pp. 228–254, 2007.

[3] A. Bemani, N. Ksairi, and M. Kountouris, "Affine frequency division multiplexing for next generation wireless communications," *IEEE Transactions on Wireless Communications*, vol. 22, no. 11, pp. 8214–8229, 2023. [Online]. Available: http://dx.doi.org/10.1109/TWC.2023.3260906

[4] H. S. Rou, G. T. F. de Abreu, J. Choi, D. González G., M. Kountouris, Y. L. Guan, and O. Gonsa, "From orthogonal time-frequency space to affine frequency-division multiplexing: A comparative study of next-generation waveforms for integrated sensing and communications in doubly dispersive channels," *IEEE Signal Processing Magazine*, vol. 41, no. 5, pp. 71–86, 2024. [Online]. Available: http://dx.doi.org/10.1109/MSP.2024.3422653

[5] W. Benzine, A. Bemani, N. Ksairi, and D. Slock, "Affine frequency division multiplexing for communications on sparse time-varying channels," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 4921–4926. [Online]. Available: http://dx.doi.org/10.1109/GLOBECOM54140.2023.10437010

[6] J. A. Tropp and A. C. Gilbert, "Signal recovery from random measurements via orthogonal matching pursuit," *IEEE Transactions on Information Theory*, vol. 53, no. 12, pp. 4655–4666, 2007.

[7] D. L. Donoho, A. Maleki, and A. Montanari, "Message-passing algorithms for compressed sensing," *Proceedings of the National Academy of Sciences*, vol. 106, no. 45, pp. 18 914–18 919, 2009.

[8] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Processing Magazine*, vol. 38, no. 2, pp. 18–44, 2021.

[9] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Model-driven deep learning for mimo detection," *IEEE Transactions on Signal Processing*, vol. 68, pp. 1702–1715, 2020. [Online]. Available: http://dx.doi.org/10.1109/TSP.2020.2976585

[10] N. Shlezinger, R. Fu, and Y. C. Eldar, "Deepsic: Deep soft interference cancellation for multiuser mimo detection," *IEEE Transactions on Wireless Communications*, vol. 20, no. 2, pp. 1349–1362, 2021. [Online]. Available: http://dx.doi.org/10.1109/TWC.2020.3032663

[11] J. Xu, L. Li, L. Zheng, and L. Liu, "Detect to learn: Structure learning with attention and decision feedback for mimo-ofdm receive processing," *IEEE Transactions on Communications*, vol. 72, no. 1, pp. 146–161, 2024. [Online]. Available: http://dx.doi.org/10.1109/TCOMM.2023.3292468

Appendices

A Alignment of Midterm MATLAB Work and Final Python Reporting

This appendix records how the midterm MATLAB materials relate to the final Python report, and clarifies the evaluation pipeline boundary.

A.1 Midterm MATLAB artifacts

The midterm project produced two primary artifacts:

1. `AFDM_UWA_acoustic.m`: The main MATLAB simulation file implementing the full AFDM-UWA chain, including BPSK mapping, DAFT-based modulation, doubly selective channel simulation, OMP support locking, and iterative MP/LS detection.
2. The interim report `2840210g_Guo_UESTC4006P_Interim_2026.pdf`: The midterm assessment document containing BER/NMSE evaluation results under the MATLAB protocol.

A.2 Why MATLAB numbers are not reused in the final report

The MATLAB and Python evaluation environments differ in the following ways:

- **Pilot format**: The Python pipeline uses a slightly different pilot embedding convention that was standardised for the learned receiver variants.
- **Test-set management**: The Python pipeline enforces strict disjoint train/test caches through the cache-based framework. The MATLAB pipeline managed random seeds manually and did not have a formal cache-based separation.
- **Logging and reproducibility**: The Python pipeline logs all evaluation parameters, random seeds, and frame-level results. The MATLAB pipeline used per-run BER averaging without the same level of provenance tracking.
- **Baseline set**: The Python pipeline evaluates multiple baselines (JOINTOMPMP, TRADPARAM FIXED, TRADPARAM CLASSICAL_FULL, CURRENTJOINT) under a common protocol. The MATLAB pipeline evaluated the traditional receiver family only.

Mixing MATLAB absolute BER values with Python results would introduce inconsistencies into the final report. The MATLAB work is therefore used as historical context for the project trajectory, not as a source of final quantitative claims.

A.3 Mapping midterm work to final results

B Full Observable Feature Vector Definitions

This appendix provides complete definitions for all 12 dimensions of the observable feature vector $\mathbf{z}_b$ used by the frozen corrector.

Table 14: Alignment of midterm MATLAB work to final Python-aligned report.

Midterm contribution	Final report treatment
Joint OMP-MP receiver design in MATLAB	Provides the historical origin for the JOINTOMPMPand TRADPARAM FIXED family in the Python codebase
MATLAB BER/NMSE evaluation results	Used as evidence of project trajectory; not used as final numerical claims
OMP support locking design decision	Carried forward to the Python pipeline; same algorithmic structure
Iterative MP/LS gain refinement	Carried forward; forms the basis of the CURRENTJOINT backbone

Table 15: Complete definition of the 12-dimensional observable feature vector. All features are extracted from the deployed receiver's iteration logs. No oracle information or side channels are used.

Idx	Symbol	Definition
1	$\ell_b^{(T)}$	Log-likelihood ratio for bit b at the final detector iteration $t = T$.
2	$\lvert\ell_b^{(T)}\rvert$	Magnitude of the final LLR; measures decision confidence.
3	$\ell_b^{(0)}$	LLR for bit b at the initial detector iteration $t = 0$.
4	$\lvert\ell_b^{(0)}\rvert$	Magnitude of the initial LLR.
5	$\ell_b^{(T)} - \ell_b^{(0)}$	Signed LLR change across the full iterative loop; encodes how beliefs evolved.
6	$\lvert\ell_b^{(T)} - \ell_b^{(0)}\rvert$	Magnitude of LLR change; encodes amount of belief revision.
7	$v_{x_b}^{(T)}$	Soft symbol estimate (posterior mean) for bit b at the final iteration.
8	$v_{x_b}^{(0)}$	Soft symbol estimate for bit b at the initial iteration.
9	$v_{x_b}^{(T)} - v_{x_b}^{(0)}$	Change in soft symbol estimate across iterations.
10	$\sigma_b^{(T)}$	Effective noise/uncertainty statistic at the final iteration (proportional to per-bit posterior variance).
11	$\sigma_b^{(0)}$	Effective noise/uncertainty statistic at the initial iteration.
12	$\mathrm{SNR}_{\mathrm{frame}}$	Frame-level SNR estimate, computed from the received power and estimated noise variance.

Implementation note. Features 1–9 are extracted directly from the iteration log maintained by the CURRENTJOINT backbone receiver. Features 10–11 are derived from the per-iteration effective noise estimates produced during the gain refinement step. Feature 12 is computed from

the pilot observations at the start of each frame. All features are normalised using statistics (mean and standard deviation per dimension) computed from the training cache before training begins.

C Paired Bootstrap Confidence Interval Methodology

This appendix describes the paired-bootstrap procedure used to compute 95 % confidence intervals for regime-level BER gain estimates.

Observation structure. For a given regime, let n denote the number of test frames in the cache. For frame i, let $e_i^{(0)}$ be the number of bit errors made by the baseline (CURRENTJOINT) and $e_i^{(1)}$ be the number of bit errors made by the corrected receiver. Define the per-frame gain as $\delta_i = e_i^{(0)} - e_i^{(1)}$. The mean gain is $\bar{\delta} = \frac{1}{n}\sum_{i=1}^{n} \delta_i$.

Bootstrap procedure. The bootstrap confidence interval for $\bar{\delta}$ is computed as follows:

1. Draw $B_{\text{boot}} = 1000$ bootstrap resamples. Each resample is obtained by drawing n frames with replacement from $\{1, \ldots, n\}$.
2. For each resample b, compute the bootstrap mean $\bar{\delta}^{(b)}$.
3. The 95 % CI is $[\hat{q}_{0.025}, \hat{q}_{0.975}]$, where $\hat{q}_\alpha$ is the α-quantile of $\{\bar{\delta}^{(1)}, \ldots, \bar{\delta}^{(B_{\text{boot}})}\}$.

The pairing (comparing baseline and corrected receiver on the same frame realisation) ensures that the CI reflects the variance in the *gain*, not the total variance of each method. This is important because the two BER values being compared are highly correlated (they are evaluated on the same channel realisations), and the unpaired CI would be substantially wider and more conservative than necessary.

Interpretation. A 95 % CI that is entirely above zero (lower bound > 0) means that the observed gain is statistically distinguishable from zero at the 5 % level under the bootstrap approximation to the sampling distribution. All four positive regimes in Table 10 satisfy this criterion. The $P_{\text{paths}} = 15$ CI is entirely below zero, confirming the statistical significance of the regression.

D Additional BER Tables

Table 16: Per-cache BER for the frozen corrector across all five evaluation regimes. Each entry is the mean BER over the frames in that cache. Values marked "—" indicate that only one cache was evaluated in that regime.

Regime	Baseline BER	Corrected BER	Gain	95 % CI
Nominal (mean of 3)	6.3553×10^{-2}	6.2493×10^{-2}	1.059×10^{-3}	$[8.42 \times 10^{-4}, 1.27 \times 10^{-3}]$
Higher Doppler	—	—	8.431×10^{-4}	$[5.70 \times 10^{-4}, 1.13 \times 10^{-3}]$
Integer-Doppler	3.0672×10^{-2}	2.9347×10^{-2}	1.325×10^{-3}	$[1.16 \times 10^{-3}, 1.50 \times 10^{-3}]$
Lower pilot power ($\rho = 0.4$)	—	—	1.670×10^{-3}	$[1.33 \times 10^{-3}, 2.03 \times 10^{-3}]$
$P_{\text{paths}} = 15$	—	—	-7.988×10^{-3}	$[-1.14 \times 10^{-2}, -4.78 \times 10^{-3}]$